

Computer Science

Chapter 4: Software

4.1 Types of software and interrupts

4.2 Types of programming language, translators and integrated development environments (IDEs)

By: Nouredine Tadjerout

Learning Objectives

4.1 Types of software and interrupts

Candidates should be able to:

- 1 Describe the difference between system software and application software and provide examples of each
- 2 Describe the role and basic functions of an operating system

Notes and guidance

- System software provides the services that the computer requires, including operating system and utility software
- Application software provides the services that the user requires
- Including:
 - managing files
 - handling interrupts
 - providing an interface
 - managing peripherals and drivers
 - managing memory
 - managing multitasking
 - providing a platform for running applications
 - providing system security
 - managing user accounts

Learning Objectives

4.1 Types of software and interrupts continued

Candidates should be able to:

- 3 Understand how hardware, firmware and an operating system are required to run applications software
- 4 Describe the role and operation of interrupts

Notes and guidance

- Applications are run on the operating system
- The operating system is run on the firmware
- The bootloader (firmware) is run on the hardware
- Including:
 - how an interrupt is generated
 - how it is handled using an interrupt service routine
 - what happens as a result of the interrupts
- Software interrupts include division by zero and two processes trying to access the same memory location
- Hardware interrupts include pressing a key on the keyboard and moving the mouse

Learning Objectives

4.2 Types of programming language, translators and integrated development environments (IDEs)

Candidates should be able to:

- 1 Explain what is meant by a high-level language and a low-level language, including the advantages and disadvantages of each
- 2 Understand that assembly language is a form of low-level language that uses mnemonics, and that an assembler is needed to translate an assembly language program into machine code
- 3 Describe the operation of a compiler and an interpreter, including how high-level language is translated by each and how errors are reported

Notes and guidance

- Advantages and disadvantages include:
 - ease of reading and writing code, e.g. low-level is hard to read
 - ease of debugging code
 - machine independence
 - direct manipulation of hardware
- A compiler translates the whole code at once before executing it, producing an executable file
- An interpreter translates and executes the code line-by-line
- A compiler provides an error report for the whole code if errors are detected
- An interpreter stops execution when an error is found

Learning Objectives

4.2 Types of programming language, translators and integrated development environments (IDEs) continued

Candidates should be able to:

- 4 Explain the advantages and disadvantages of a compiler and an interpreter
- 5 Explain the role of an IDE in writing program code and the common functions IDEs provide

Notes and guidance

- To include an understanding that an interpreter is mostly used when developing a program and a compiler is used to translate the final program
- Including:
 - code editors
 - run-time environment
 - translators
 - error diagnostics
 - auto-completion
 - auto-correction
 - prettyprint

Resources



Craig 'n' Dave For Students

CRAIG 'N' DAVE

To explore the entire series,
visit <https://craigndave.org/free-videos/>

Resources



To explore the entire series,
visit www.hoddereducation.com/cambridge-igcse-computerscience

Learning Objectives

4.1 Types of software and interrupts

Candidates should be able to:

- 1 Describe the difference between system software and application software and provide examples of each
- 2 Describe the role and basic functions of an operating system

Notes and guidance

- System software provides the services that the computer requires, including operating system and utility software
- Application software provides the services that the user requires
- Including:
 - managing files
 - handling interrupts
 - providing an interface
 - managing peripherals and drivers
 - managing memory
 - managing multitasking
 - providing a platform for running applications
 - providing system security
 - managing user accounts

4.1 Types of software and interrupts

Watch the video for System software and application software

<https://www.youtube.com/watch?v=MIMNcWMt4iU>



4.1 Types of software and interrupts

4.1.1 System software and application software

Hardware and software



All physical parts of a computer system and its related devices, both internal or external – e.g., motherboard, hard drive, RAM, monitor, keyboard, mouse, printer, etc.

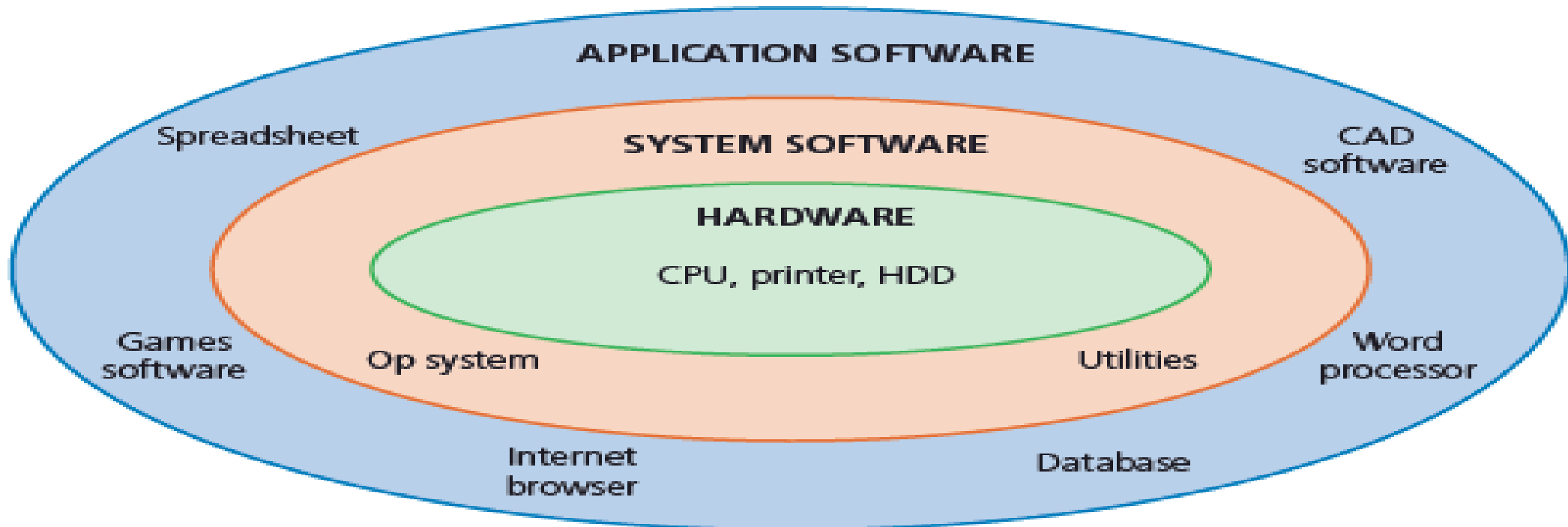


General term covering all computer programs. Most software falls into two broad categories – system and application.

4.1 Types of software and interrupts

4.1.1 System software and application software

All computers begin life as a group of connected hardware items. Without software, the hardware items would be useless. This section considers the link between hardware and software. Figure 4.1 summarises the hierarchy of software and hardware.

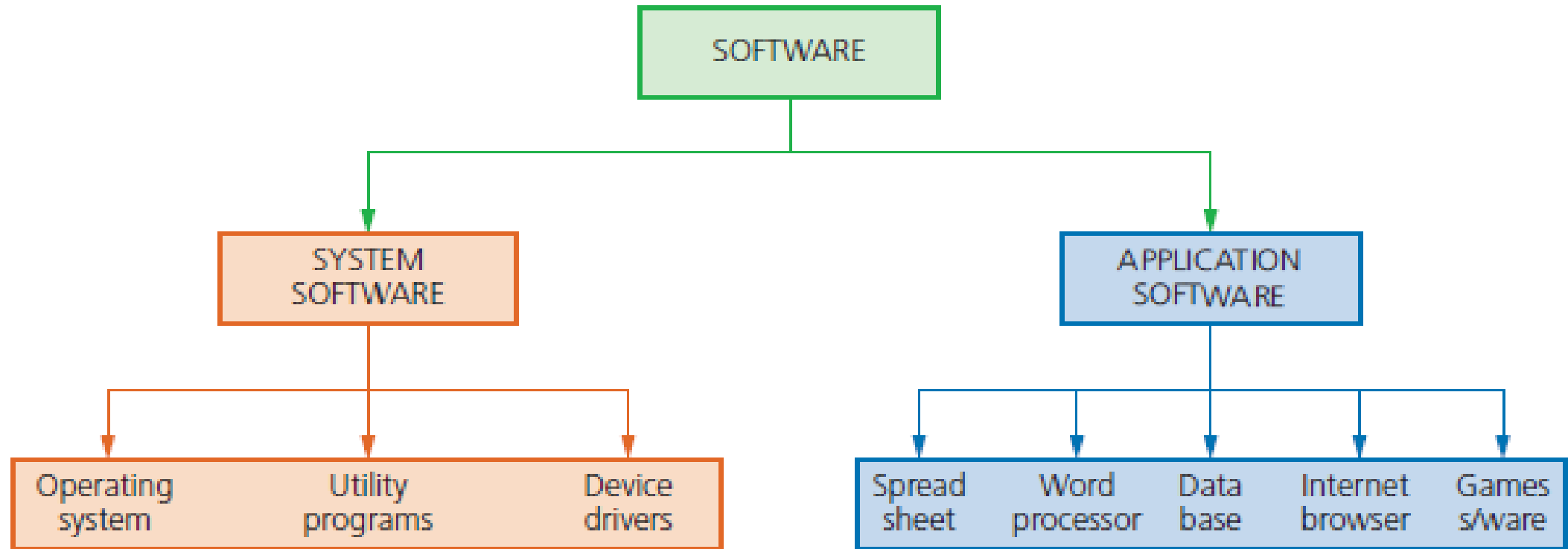


▲ **Figure 4.1** Software and hardware hierarchy

4.1 Types of software and interrupts

4.1.1 System software and application software

You will notice from Figure 4.1 that there are two types of software: **system software** and **application software**:



▲ **Figure 4.2** Software types

4.1 Types of software and interrupts

4.1.1 System software and application software

General features of system software:

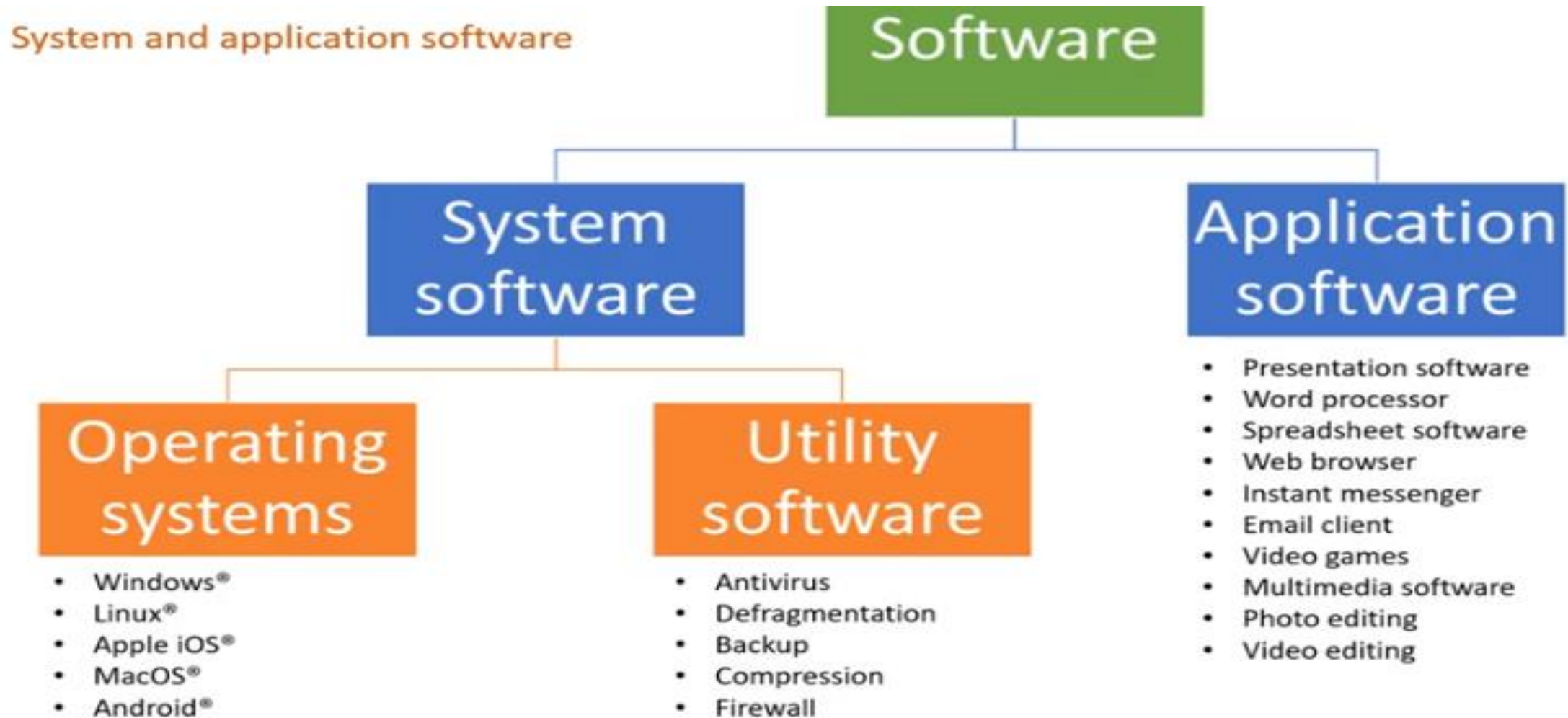
- » set of programs to control and manage the operation of computer hardware
- » provides a platform on which other software can run
- » required to allow hardware and software to run without problems
- » provides a human computer interface (HCI)
- » controls the allocation and usage of hardware resources.

General features of application software:

- » used to perform various applications (apps) on a computer
- » allows a user to perform specific tasks using the computer's resources
- » may be a single program (for example, NotePad) or a suite of programs (for example, Microsoft Office)
- » user can execute the software as and when they require.

4.1 Types of software and interrupts

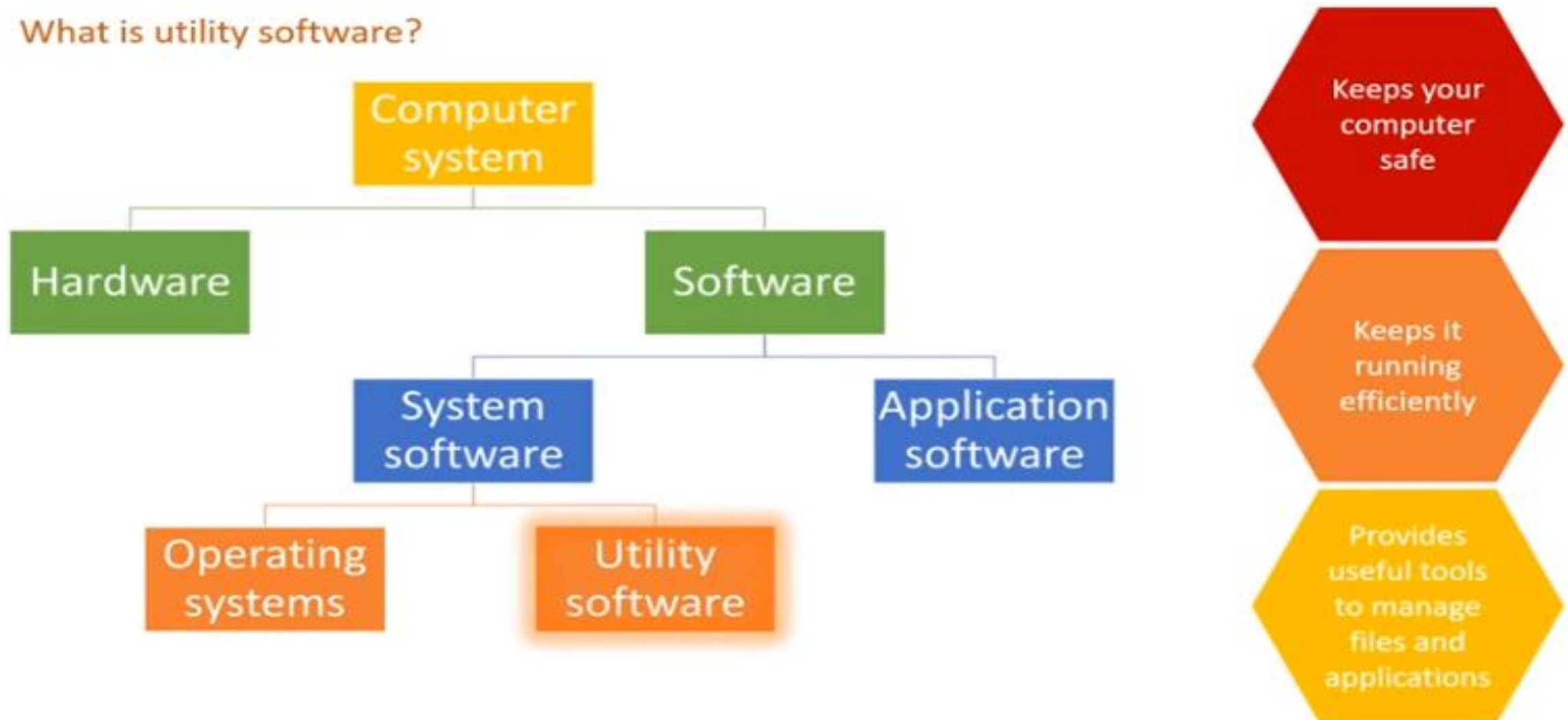
4.1.1 System software and application software



4.1 Types of software and interrupts

4.1.1 System software and application software

What is utility software?



4.1 Types of software and interrupts

4.1.1 System software and application software



4.1 Types of software and interrupts

4.1.1 System software and application software

Utility software: File repair



File repair software attempts to correct issues and restore a file to its original working state.

Often built into applications, as well as being a separate, dedicated software tool found in most operating systems.

4.1 Types of software and interrupts

4.1.1 System software and application software

Utility software: Backup



Backups can be set up to be manual, automatic or scheduled.

You can perform **full backups** or **incremental backups**.

You can **back up** to different media or an off-site cloud system.

Backup software often comes with an operating system but can be purchased separately.

4.1 Types of software and interrupts

4.1.1 System software and application software

Utility software: Data compression

11111111 00000000 11111111 00000000



Compression reduces the size of a file so that it takes up less space and downloads faster over the internet.

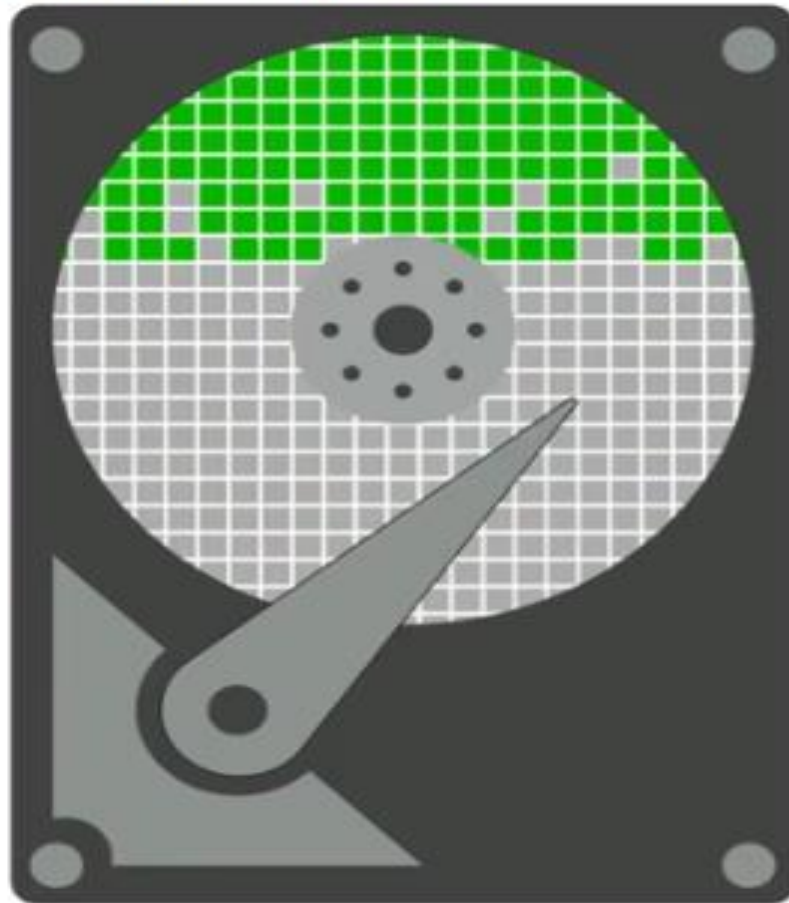
Compressed files must be extracted before they can be read.

Depending on the algorithm used, data is either lost, reducing the quality of an image or audio file (e.g., JPEG, MP3), or represented in a different way using binary, retaining the original data in a new, compressed format (e.g., ZIP).

4.1 Types of software and interrupts

4.1.1 System software and application software

Utility software: Defragmentation



Defragmentation reorganises files on a hard disk, putting fragments of files and free space back together.

Reduces the movement of the read/write head across the surface of the disk, which speeds up file access.

Solid-state drives should not be defragmented, as they have no moving parts. It is unnecessary and reduces the drive's lifespan.

4.1 Types of software and interrupts

4.1.1 System software and application software

Utility software: Anti-malware



Anti-malware software helps keep your computer and files safe from many types of malware including viruses, trojans, worms and spyware

Most operating systems come with malware protection built in, configured and turned on.

Like many other forms of **utility** software, there are companies who specialise in dedicated **anti-malware** programs.

4.1 Types of software and interrupts

4.1.1 System software and application software



System software and application software

All computer systems are comprised of both hardware and software.

- Hardware: All physical parts of a computer system and its related devices, both internal and external.
- Software: General term covering all computer programs.

Most software falls into two broad categories.

- System software: Provides the services required by the computer.
- Application software: Provides the services required by the user.

System software is broken down into two further categories.

- Operating system: Allows applications to interface with hardware.
- Utility software: Helps keep a computing device secure and in good working order.

Application software covers all other software – e.g., productivity, communication, entertainment, etc.

Application software runs on the operating system which, in turn, talks to hardware and other applications.

4.1 Types of software and interrupts

4.1.2 Operating systems

Watch the video for Operating systems

<https://www.youtube.com/watch?v=4HyZd3X3d5c>



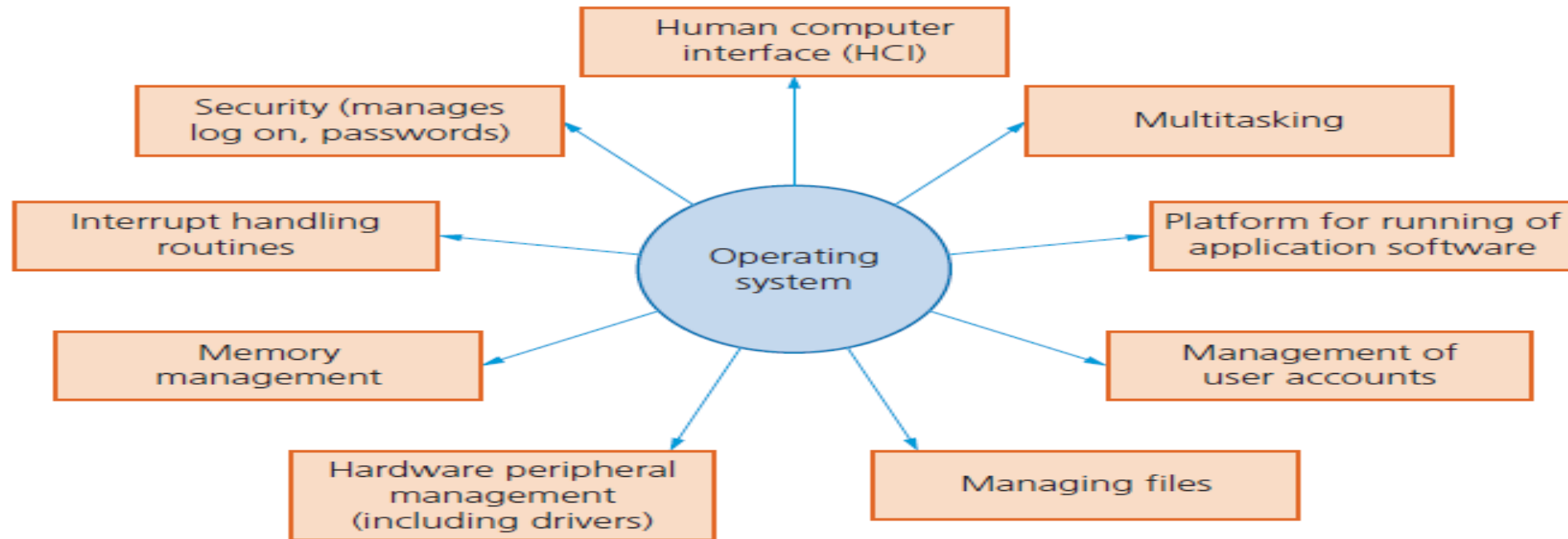
4.1 Types of software and interrupts

4.1.2 Operating systems

To enable computer systems to function correctly and allow users to communicate with computer systems, software known as an **operating system** needs to be installed. An operating system provides both the environment in which applications can be run and a useable interface between humans and computer.

An operating system also disguises the complexity of computer software and hardware. Common examples of operating systems include: Microsoft Windows, Apple Mac OS, Google Android and Apple IOS (the latter two being used primarily on tablets and smartphones). Most computers store the operating system on a hard disk drive (HDD) or solid state drive (SSD) since they tend to be very large programs. Mobile phones and tablets store the operating system on a solid state device since they are too small to accommodate an HDD.

Figure 4.7 summarises some of the functions in a typical operating system.



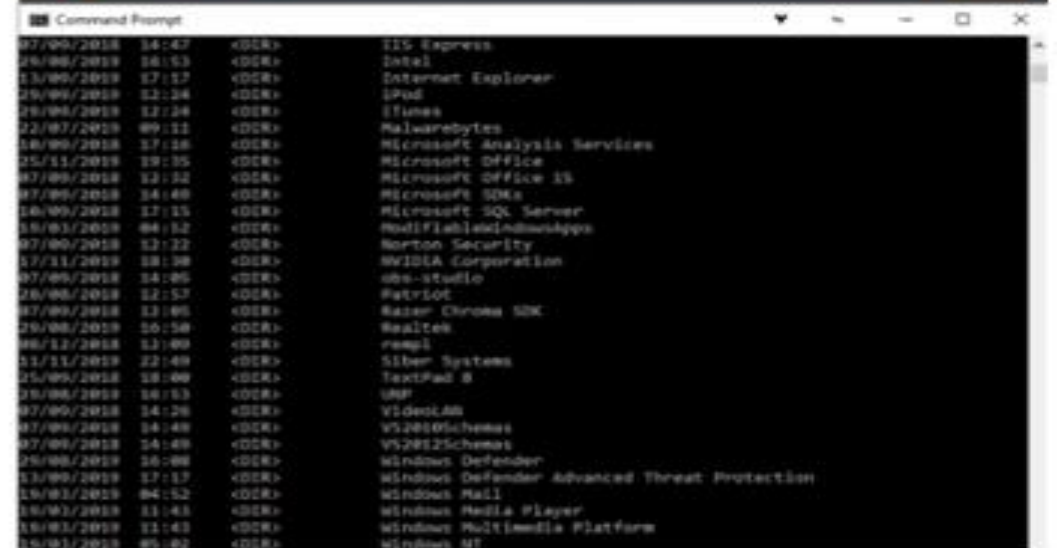
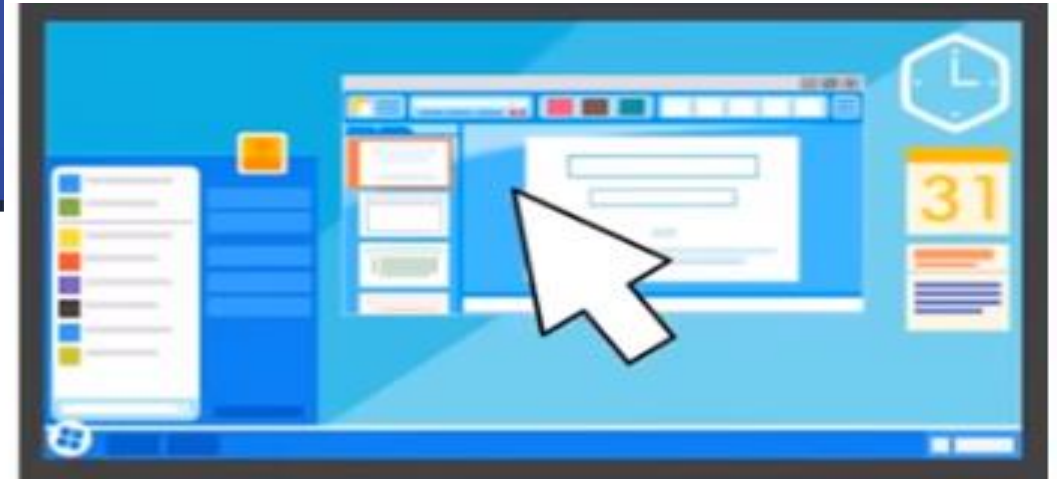
▲ Figure 4.7 Operating system functions

4.1 Types of software and interrupts

4.1.2 Operating systems

User interfaces

- Graphical (GUI)
- Command line
- Menu-driven
- Natural language



4.1 Types of software and interrupts

4.1.2 Operating systems

handling interrupts

Multitasking

Multitasking is when you have more than one program open and running at the same time.

The processor allocates a small amount of time to each process and cycles between them.

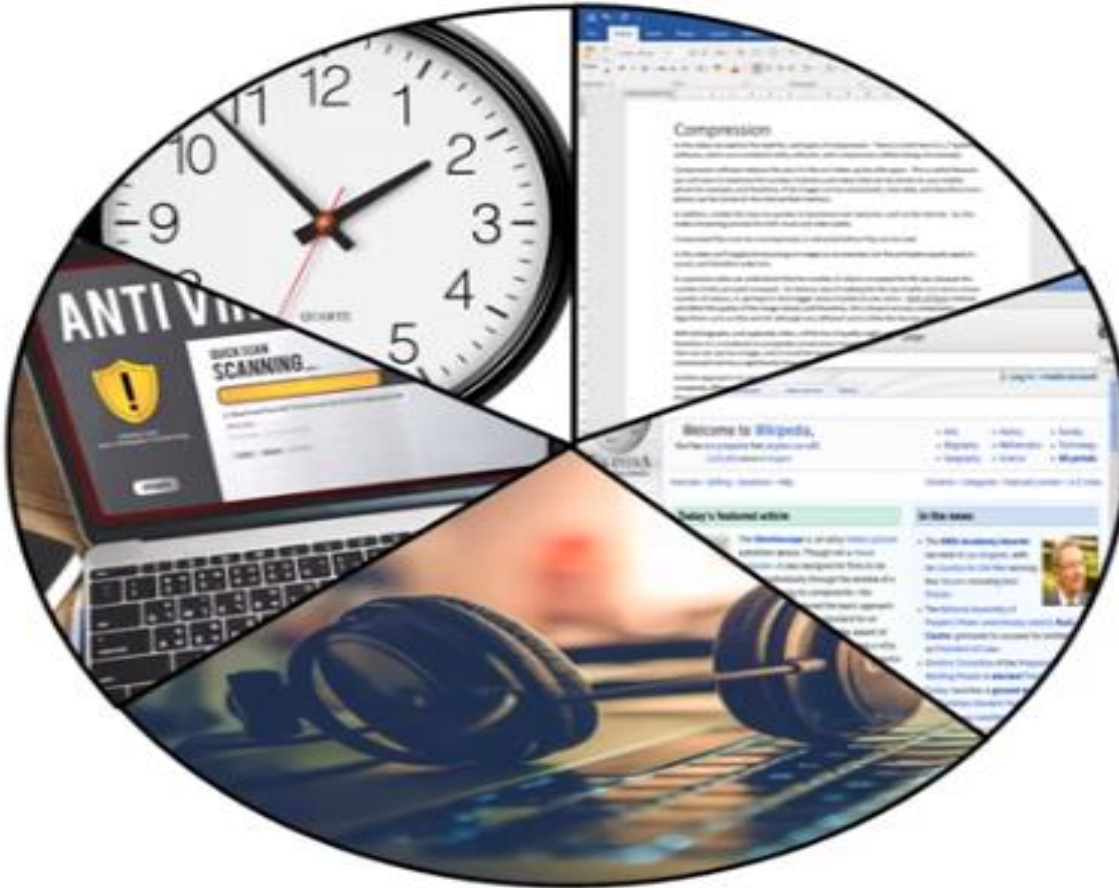
As this happens so quickly, it appears as if multiple programs are executing simultaneously.



4.1 Types of software and interrupts

4.1.2 Operating systems managing multitasking

Resource management/multitasking



- We are working on a document in a word processor...
 - ...while at the same time, doing some research in a web browser...
 - ...streaming music...
 - ...and running an antivirus program in the background.
 - All the while, our system clock is updating, and our **operating system** is performing various background tasks.
-
- It appears as if all the tasks are operating simultaneously.
 - The **operating system** is assigning a time slice to each task, letting it process and then moving on to the next one so quickly that it all appears to be happening at the same time.

4.1 Types of software and interrupts

4.1.2 Operating systems

managing peripherals and drivers

Device drivers

Your computer needs to be able to output to a wide range of devices.

A printed document should look the same regardless of the make or model of printer.

However, the technology behind each printer is very different. To overcome this inconsistency, we use device drivers.

Printer drivers translate the operating system instructions to print a document into a series of bespoke instructions that the printer will understand.



4.1 Types of software and interrupts

4.1.2 Operating systems



Operating systems

Multitasking

- Multitasking enables your computer to run several applications at the same time by giving each application a small slice of processor time and cycling between them.
- This process allows more than one program to be held in memory at the same time, as well as any data being shared between them (e.g., copy and paste).
- For example, you can listen to music on your PC at the same time as using a word processor.

Memory management

- When a program is loaded, the operating system decides where to hold it in memory.
- Over time, as more and more programs are opened and closed, the memory becomes fragmented because programs use different amounts of memory – the operating system must keep track of these program fragments.
- When main memory (RAM) is full, the operating system will fall back to virtual memory.

Device drivers

- Translate operating system instructions into commands that specific hardware will understand.
- Each peripheral needs a device driver – thankfully, many are already built into the operating system.

4.1 Types of software and interrupts

4.1.2 Operating systems

Watch the video for Operating systems

<https://www.youtube.com/watch?v=AlmENd2McGk>



4.1 Types of software and interrupts

4.1.2 Operating systems

managing user accounts and security



User management: Allows multiple users to log into the same computer.

The operating system will retain settings for each user such as icons, desktop backgrounds, etc.

Each user may have different access rights to files and programs.

A client-server network may impose a fixed or roaming profile for a user and manage login requests to the network.

4.1 Types of software and interrupts

4.1.2 Operating systems managing files

File management



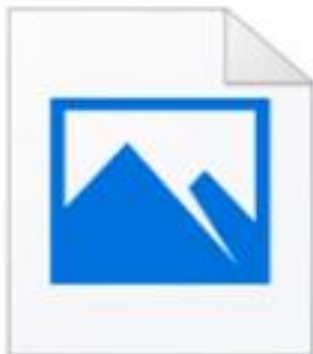
Film.avi



Spreadsheet.xls



Essay.docx



Cat.bmp



My holiday.pptx



Poster.pubx

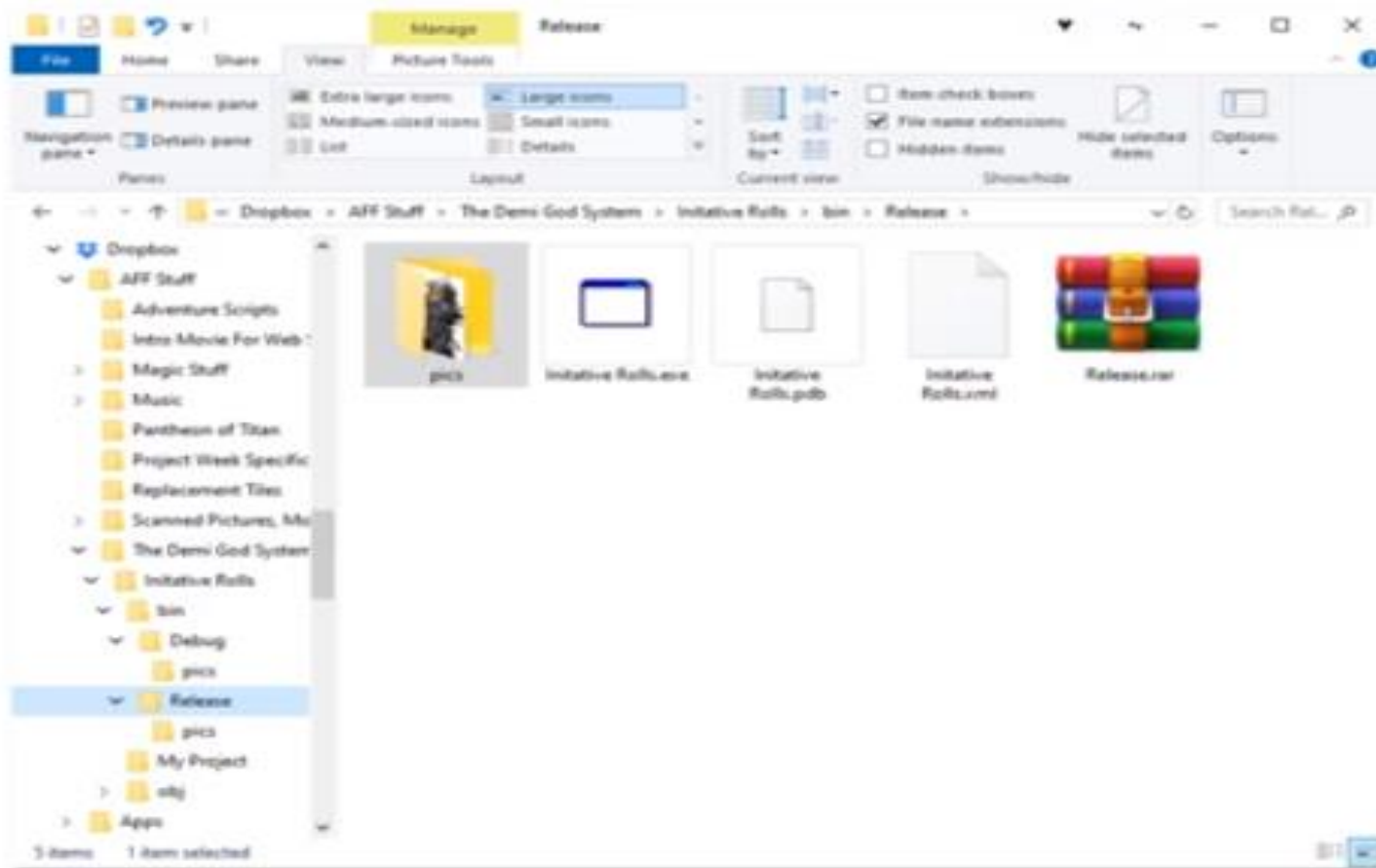
Data is stored in files.

An extension to the filename tells the operating system which application to load the file into.

4.1 Types of software and interrupts

4.1.2 Operating systems managing files

File management



The **operating system** presents a logical structure of files in folders and allows the user to rename, delete, copy and move files.

4.1 Types of software and interrupts

4.1.2 Operating systems



Operating systems

User management/security

- Allowing different users to log into their own account with a username and password.
- Each account can be granted different levels of access depending on personal needs/security policies.
- The **operating system** can log user access, who logs on and off, when and from which device, as well as logging them off after a set length of time.

File management

- Create files/folders
- Delete files/folder
- Move and organise files/folders
- Copy files/folders
- Easily search for files
- Restore deleted files
- Resolve file conflicts between users

4.1 Types of software and interrupts

4.1.2 Operating systems

Watch the video for the purpose and functions of operating systems

<https://www.youtube.com/watch?v=cgiHpRPqCpo>



4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

The function of operating systems



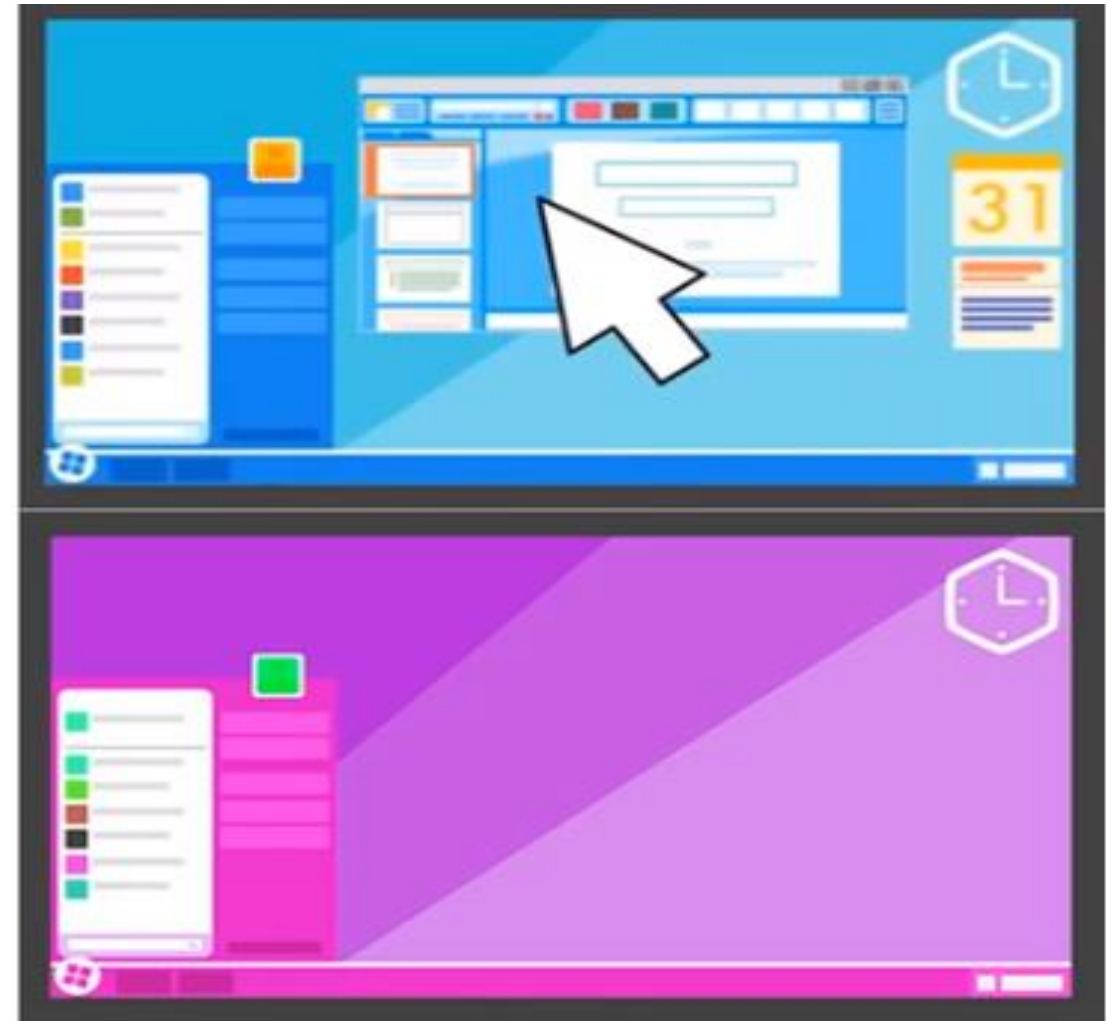
4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

User interface: Graphical user interface (GUI)

- Windows, icons, menus, pointers (WIMP)
- Visual
- Interactive
- Intuitive
- Optimised for mouse and touch gesture input



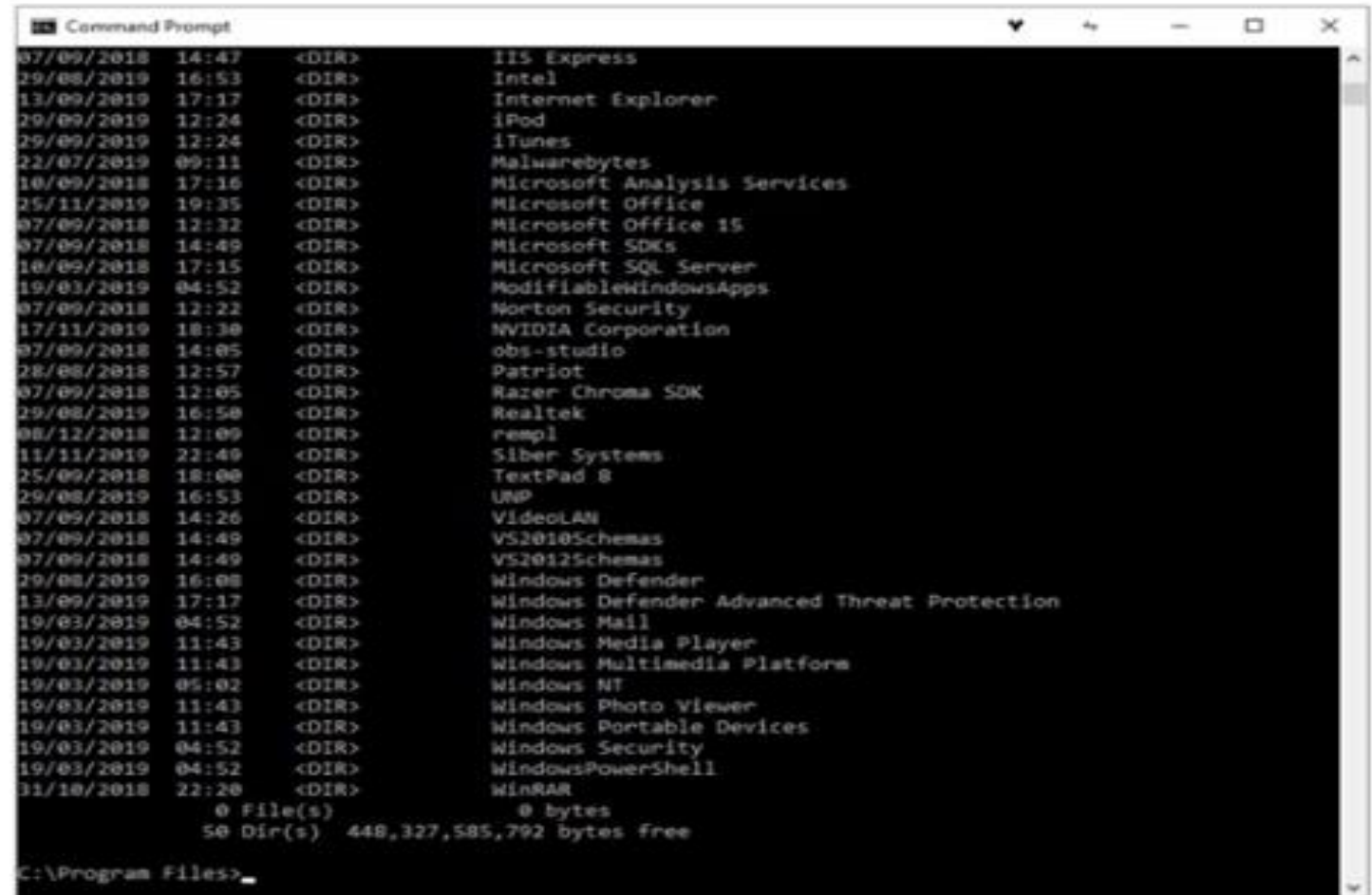
4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

User interface: Command line

- Text-based
- Less resource-intensive than a GUI
- More commands than a GUI
- Highly efficient
- Best suited for advanced users
- Used to automate processes with scripts



```
Command Prompt
07/09/2018 14:47 <DIR> IIS Express
29/08/2019 16:53 <DIR> Intel
13/09/2019 17:17 <DIR> Internet Explorer
29/09/2019 12:24 <DIR> iPod
29/09/2019 12:24 <DIR> iTunes
22/07/2019 09:11 <DIR> Malwarebytes
10/09/2018 17:16 <DIR> Microsoft Analysis Services
25/11/2019 19:35 <DIR> Microsoft Office
07/09/2018 12:32 <DIR> Microsoft Office 15
07/09/2018 14:49 <DIR> Microsoft SDKs
10/09/2018 17:15 <DIR> Microsoft SQL Server
19/03/2019 04:52 <DIR> ModifiableWindowsApps
07/09/2018 12:22 <DIR> Norton Security
17/11/2019 18:30 <DIR> NVIDIA Corporation
07/09/2018 14:05 <DIR> obs-studio
28/08/2018 12:57 <DIR> Patriot
07/09/2018 12:05 <DIR> Razer Chroma SDK
29/08/2019 16:50 <DIR> Realtek
08/12/2018 12:09 <DIR> repl
11/11/2019 22:49 <DIR> Siber Systems
25/09/2018 18:00 <DIR> TextPad 8
29/08/2019 16:53 <DIR> UWP
07/09/2018 14:26 <DIR> VideoLAN
07/09/2018 14:49 <DIR> VS2010Schemas
07/09/2018 14:49 <DIR> VS2012Schemas
29/08/2019 16:08 <DIR> Windows Defender
13/09/2019 17:17 <DIR> Windows Defender Advanced Threat Protection
19/03/2019 04:52 <DIR> Windows Mail
19/03/2019 11:43 <DIR> Windows Media Player
19/03/2019 11:43 <DIR> Windows Multimedia Platform
19/03/2019 05:02 <DIR> Windows NT
19/03/2019 11:43 <DIR> Windows Photo Viewer
19/03/2019 11:43 <DIR> Windows Portable Devices
19/03/2019 04:52 <DIR> Windows Security
19/03/2019 04:52 <DIR> WindowsPowerShell
31/10/2018 22:20 <DIR> WinRAR
0 File(s) 0 bytes
50 Dir(s) 448,327,585,792 bytes free
C:\Program Files>
```

4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

User interface: Menu

- Successive menus presented to the user
- Single options chosen at each stage
- Often with buttons on a keypad



4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

User interface: Natural language

- Responds to questions in spoken language.
- Not always reliable but improving all the time.



4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems

Differences between GUI and CLI interfaces

Interface	Advantages	Disadvantages
command line interface (CLI)	- the user is in direct communication with the computer - the user is not restricted to a number of pre-determined options - it is possible to alter computer configuration settings - uses a small amount of computer memory	- the user needs to learn a number of commands to carry out basic operations - all commands need to be typed in which takes time and can be error-prone - each command must be typed in using the correct format, spelling, and so on
graphical user interface (GUI)	- the user doesn't need to learn any commands - it is more user-friendly; icons are used to represent applications - a pointing device (such as a mouse) is used to click on an icon to launch the application – this is simpler than typing in commands or a touch screen can be used where applications are chosen by simply touching the icon on the screen	- this type of interface uses up considerably more computer memory than a CLI interface - the user is limited to the icons provided on the screen - needs an operating system, such as Windows, to operate, which uses up considerable memory

4.1 Types of software and interrupts

4.1.2 Operating systems

The purpose and functions of operating systems



The purpose and functions of operating systems

An **operating system** provides an interface between the user and the hardware of a computing device. It provides a platform for **application software** to run.

The **operating system** performs vital roles, such as:

- File management
- Interrupt handling
- User interface
- Peripheral and driver management
- Memory management
- Multitasking
- System security
- User account management

Operating systems come with a variety of different interfaces for user interaction including:

- Graphical user interface (GUI)
- Command line
- Menu-based
- Natural language

4.1 Types of software and interrupts

4.1.3 Running of applications

Watch the video for Interrupts

<https://www.youtube.com/watch?v=DYaXpfyp2II>



4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

An **interrupt** is a signal sent from a device or from software to the microprocessor. This will cause the microprocessor to temporarily stop what it is doing so that it can service the interrupt. Interrupts can be caused by:

- » a timing signal
- » an input/output process (for example, a disk drive or printer requiring more data)
- » a hardware fault (for example, a paper jam in the printer)
- » user interaction (for example, the user presses a key (or keys) on a keyboard, such as <CTRL><ALT><BREAK>, which causes the system to be interrupted)
- » software errors that cause a problem (for example, an .exe file that cannot be found to initiate the execution of a program, two processes trying to access the same memory location, or an attempt to divide by zero).

-Once the interrupt signal is received, the microprocessor either carries on with what it was doing or stops to service the device or program that caused the interrupt. The computer needs to identify the interrupt type and also establish the level of **interrupt priority**.

-Interrupts allow computers to carry out many tasks or to have several windows open at the same time. An example would be downloading a file from the internet at the same time as listening to some music from a library.

Whenever an interrupt is received it needs to be **serviced**. The status of the current task being run first needs to be saved.

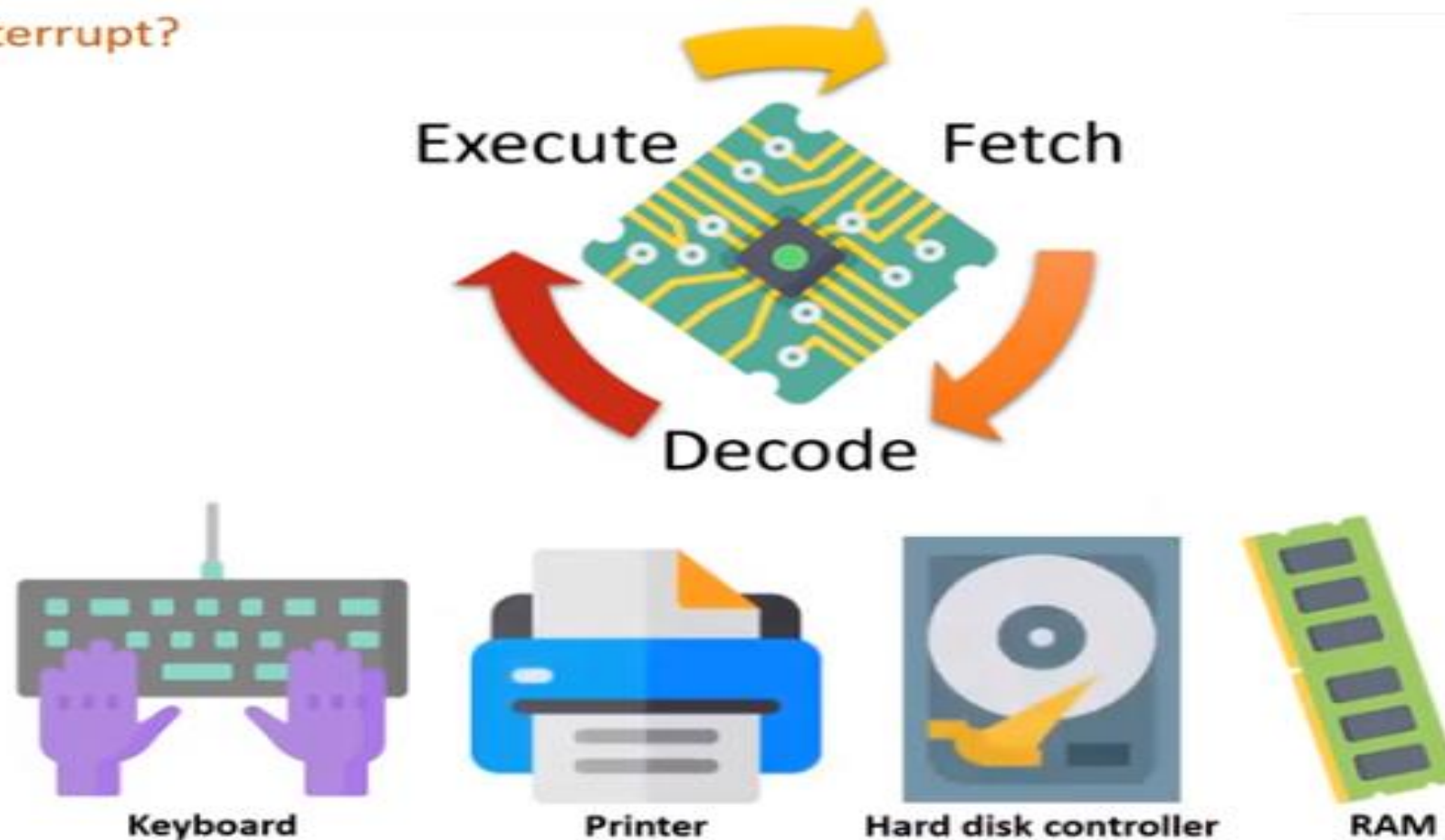
-The contents of the Program Counter (PC) and other registers are saved. Then the **interrupt service routine (ISR)** is executed by loading the start address into the Program Counter (PC). Once the interrupt has been fully serviced, the status of the interrupted task is reinstated (the contents of all the saved registers are then retrieved) and the process continues.

4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

What is an interrupt?

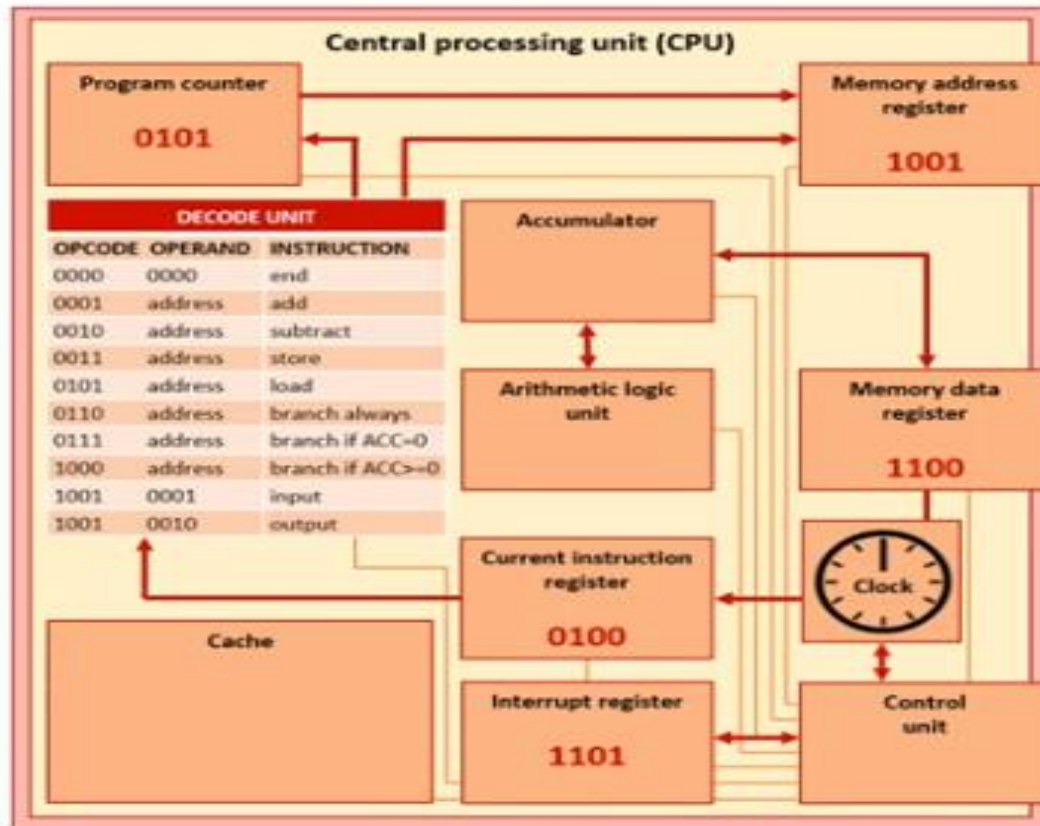


4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

How are interrupts handled?



Remember, the CPU contains registers that hold data currently being handled including:

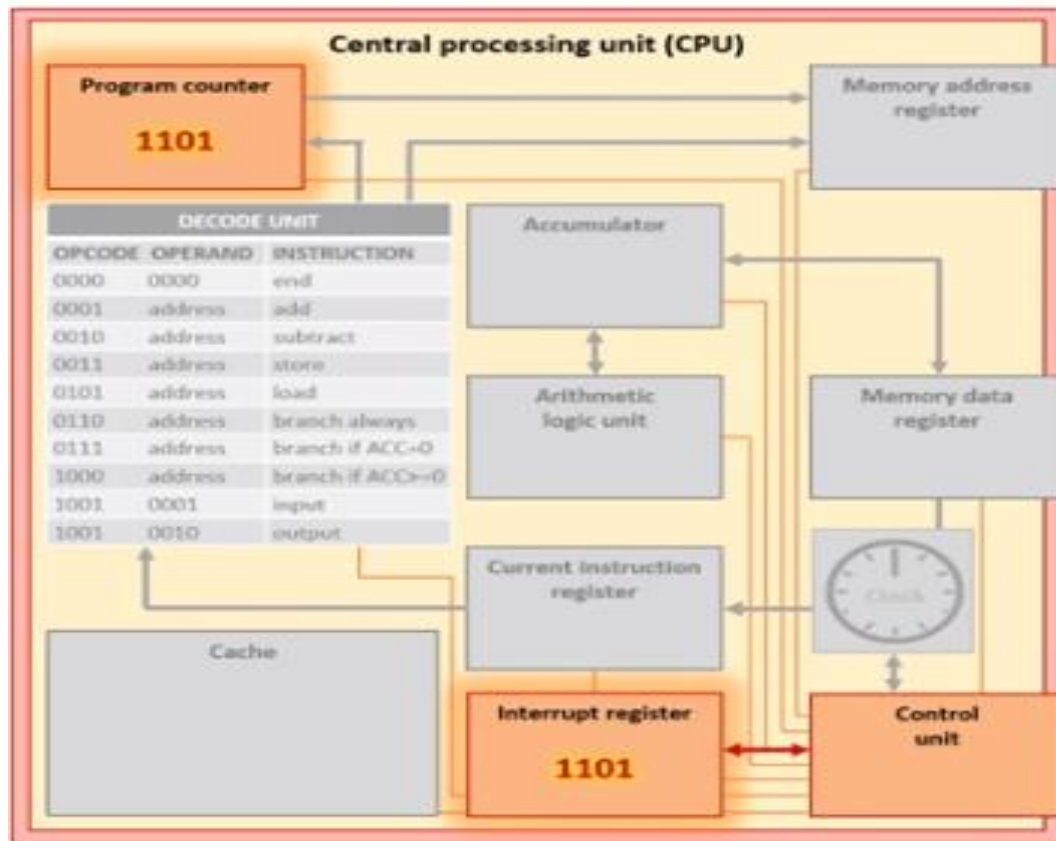
- **Program counter:** Memory location of the next instruction to be executed.
- **Current instruction register:** The current instruction being executed.
- Various other registers (**MAR/MDR**) and general-purpose registers.

4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

How are interrupts handled?



The **interrupt service routine** is simply a program with a set of instructions that need to be fetched, decoded and executed to carry out the operations of the **interrupt**.

That means the contents of the **program counter** need to be changed to point to the address for the first instruction of the **interrupt**.

When the interrupt is complete, how will the processor know to continue the previously executing program?

4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

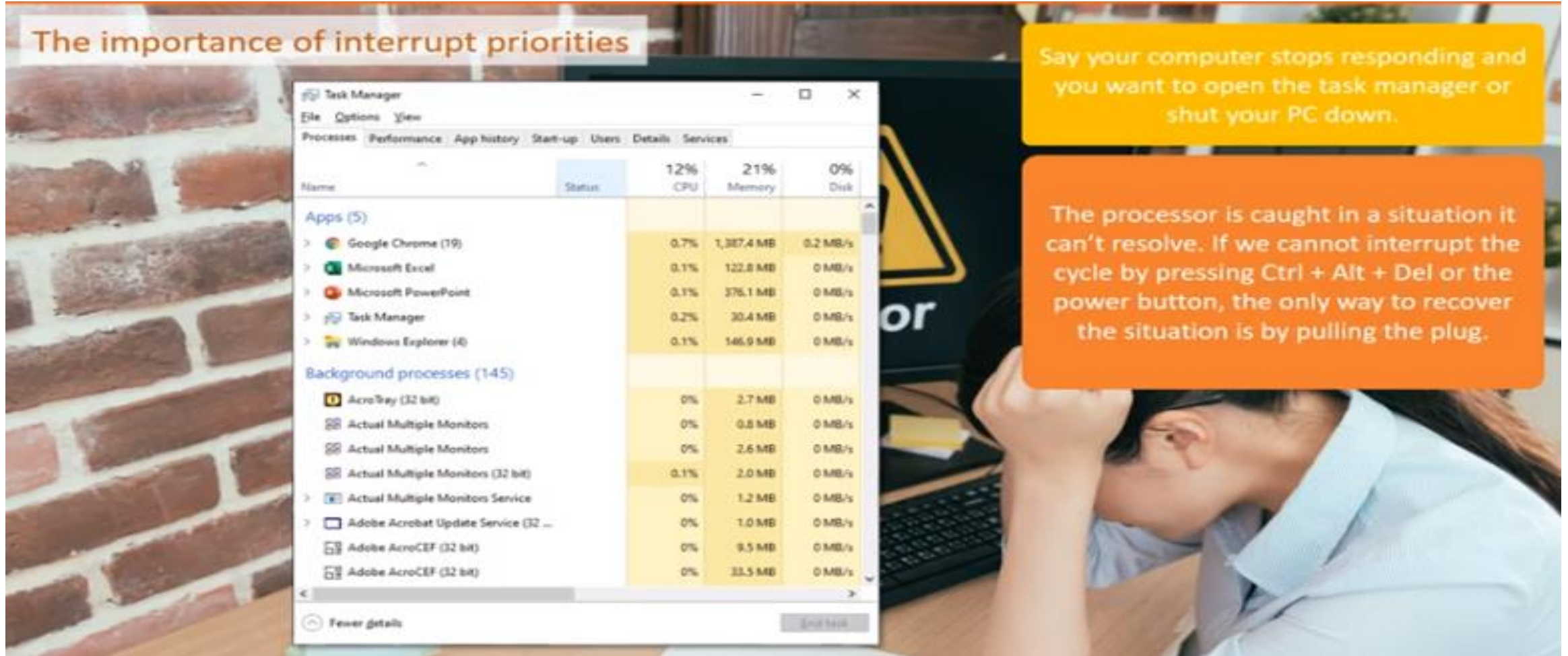


4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

The importance of interrupt priorities



Name	Status	12% CPU	21% Memory	0% Disk
Apps (5)				
Google Chrome (19)		0.7%	1,387.4 MB	0.2 MB/s
Microsoft Excel		0.1%	122.8 MB	0 MB/s
Microsoft PowerPoint		0.1%	376.1 MB	0 MB/s
Task Manager		0.2%	30.4 MB	0 MB/s
Windows Explorer (4)		0.1%	146.9 MB	0 MB/s
Background processes (145)				
AcroTray (32 bit)		0%	2.7 MB	0 MB/s
Actual Multiple Monitors		0%	0.8 MB	0 MB/s
Actual Multiple Monitors		0%	2.6 MB	0 MB/s
Actual Multiple Monitors (32 bit)		0.1%	2.0 MB	0 MB/s
Actual Multiple Monitors Service		0%	1.2 MB	0 MB/s
Adobe Acrobat Update Service (32 ...)		0%	1.0 MB	0 MB/s
Adobe Acrobat (32 bit)		0%	9.5 MB	0 MB/s
Adobe Acrobat (32 bit)		0%	33.5 MB	0 MB/s

Say your computer stops responding and you want to open the task manager or shut your PC down.

The processor is caught in a situation it can't resolve. If we cannot interrupt the cycle by pressing Ctrl + Alt + Del or the power button, the only way to recover the situation is by pulling the plug.

4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts

Interrupt priorities

Interrupt	Examples
Hardware	• Power supply failure. • Power/reset button pressed. • Power-down command (screen off, hibernate).
User	• Moving the mouse. • Clicking/tapping an icon to open a new program. • Keyboard presses.
Software	• Illegal instruction encountered. • Arithmetic overflow. • New login request.
Timer	• Data-logging program reading an input sensor every second. • Screen recording application.
Input/output devices	• Buffer nearly empty. • Signal the completion of a data transfer to/from a device. • Printer ink supply notification.

4.1 Types of software and interrupts

4.1.3 Running of applications

Interrupts



An **interrupt** is a request for the processor's attention. **Interrupts** can be generated by either software or hardware.

- Software **interrupts** include division-by-zero and two processes trying to access the same memory location.
- Hardware **interrupts** include pressing a key on the keyboard and moving the mouse.

Interrupts are placed into an **interrupt register**. The **interrupt register** is checked after each fetch-decode-execute cycle.

If an **interrupt** is detected and is higher priority than what is currently being executed, then control is passed to the **interrupt service routine**.

The current contents of the registers are stored away so the program being interrupted can be resumed later.

Learning Objectives

4.1 Types of software and interrupts continued

Candidates should be able to:

- 3 Understand how hardware, firmware and an operating system are required to run applications software
- 4 Describe the role and operation of interrupts

Notes and guidance

- Applications are run on the operating system
- The operating system is run on the firmware
- The bootloader (firmware) is run on the hardware
- Including:
 - how an interrupt is generated
 - how it is handled using an interrupt service routine
 - what happens as a result of the interrupts
- Software interrupts include division by zero and two processes trying to access the same memory location
- Hardware interrupts include pressing a key on the keyboard and moving the mouse

4.1 Types of software and interrupts

4.1.3 Running of applications

Watch the video for Relationship between hardware, firmware and operating systems

https://www.youtube.com/watch?v=W2mwbb9_7D8



4.1 Types of software and interrupts

4.1.3 Running of applications

Relationship between hardware, firmware and operating systems



Hardware, firmware and the operating system

Applications are run on the **operating system**.

The **operating system** is run on **firmware**.

The **bootloader (firmware)** is run on the **hardware**.

The **bootloader** is located in **ROM** (non-volatile) storage and is one of the first programs loaded into memory when a computer is turned on.

You can think of this like a hierarchy with the user at the top and hardware at the bottom:

- User → Application software → Operating system → Firmware → Hardware

4.1 Types of software and interrupts

4.1.3 Running of applications

Relationship between hardware, firmware and operating systems

When a computer starts up, part of the operating system needs to be loaded into RAM – this is known as **booting up** the computer (or a **bootstrap loader**).

The start-up of the computer's motherboard is handled by the basic input/output system (BIOS). The BIOS tells the computer where the storage device that holds the operating system can be found; it then loads the part of the operating system that is needed and executes it.

The BIOS is often referred to as **firmware**. Firmware is defined as a program that provides low level control for devices.

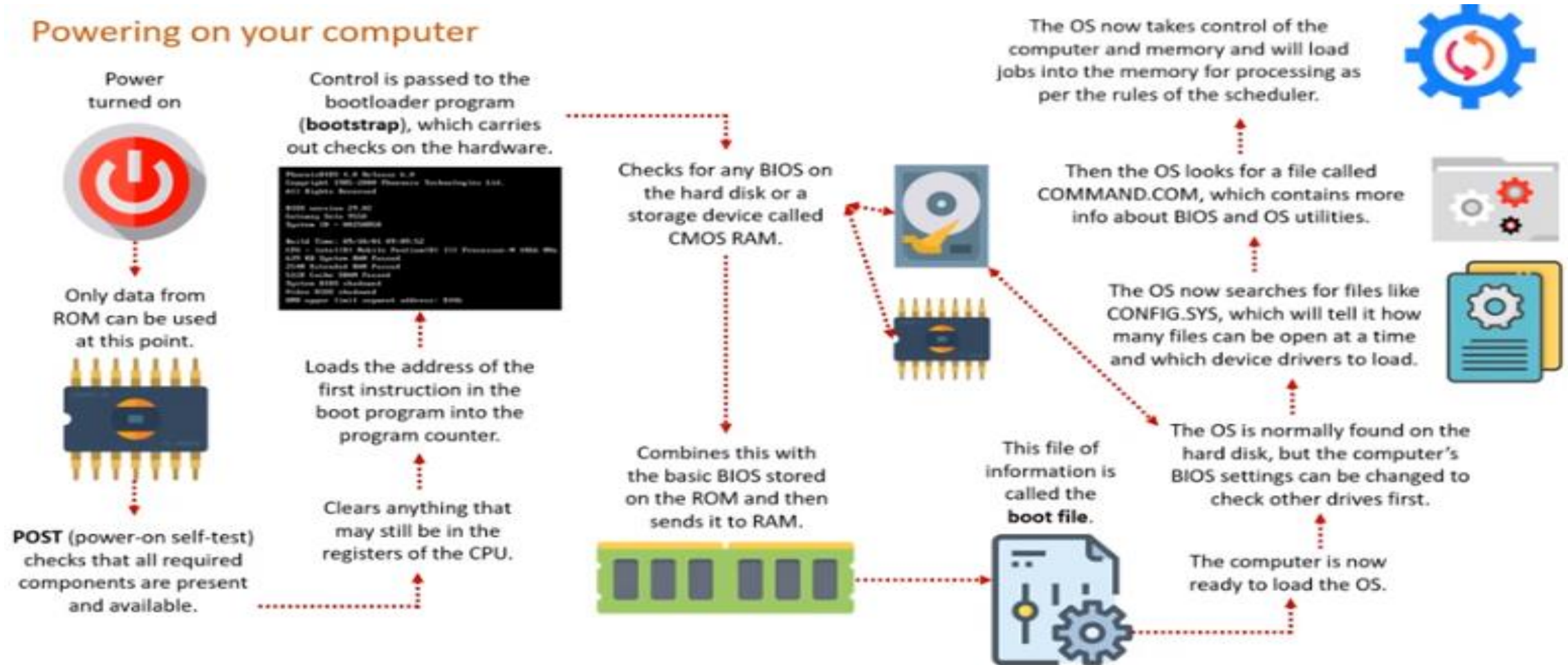
The BIOS program is stored in a special type of ROM, called an **EEPROM** (Electrically Erasable Programmable ROM). EEPROM is a flash memory chip, which means its contents remain even when the computer is powered down. However, it also means the BIOS can be rewritten, updated or even deleted by a user.

However, while the BIOS is stored on an EEPROM, the BIOS **settings** are stored on a CMOS chip (Complementary Metal Oxide Semi-conductor). The CMOS is powered up at all times via a rechargeable battery on the motherboard. Therefore, the BIOS settings would be reset if the battery was removed or disconnected for some reason. Once the CMOS is re-started, it will access the same BIOS program from EEPROM, but the settings will now be the default factory settings. Consequently, if a user has changed the BIOS settings (for example, the clock speed), the settings will revert to those settings made at the factory once power is restored to the CMOS.

4.1 Types of software and interrupts

4.1.3 Running of applications

Relationship between hardware, firmware and operating systems



4.1 Types of software and interrupts

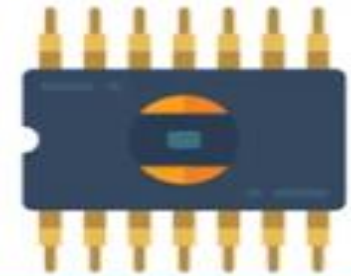
4.1.3 Running of applications

Relationship between hardware, firmware and operating systems

The evolution of ROM

All types are non-volatile. All data is either:

- Unchangeable (read-only).
- Changeable but requires a special operation to do so.



ROM	PROM	EPROM	EEPROM
Read-only memory	Programmable read-only memory	Erasable programmable read-only memory	Electrically erasable programmable read-only memory
• Programming of data must be complete when the chip is created. • Cannot be reprogrammed or rewritten.	• Blank when initially manufactured. • Can be purchased and coded once with a special tool known as a programmer – known as burning the PROM.	• Can be rewritten many times. • Erasing requires a special tool that emits a certain frequency of UV light. • Must be removed from a device before it can be erased and reprogrammed.	• The chip does not have to be removed to be rewritten. • The entire chip does not need to be completely erased to change a specific portion of it. • Changing contents does not require specialist equipment.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages

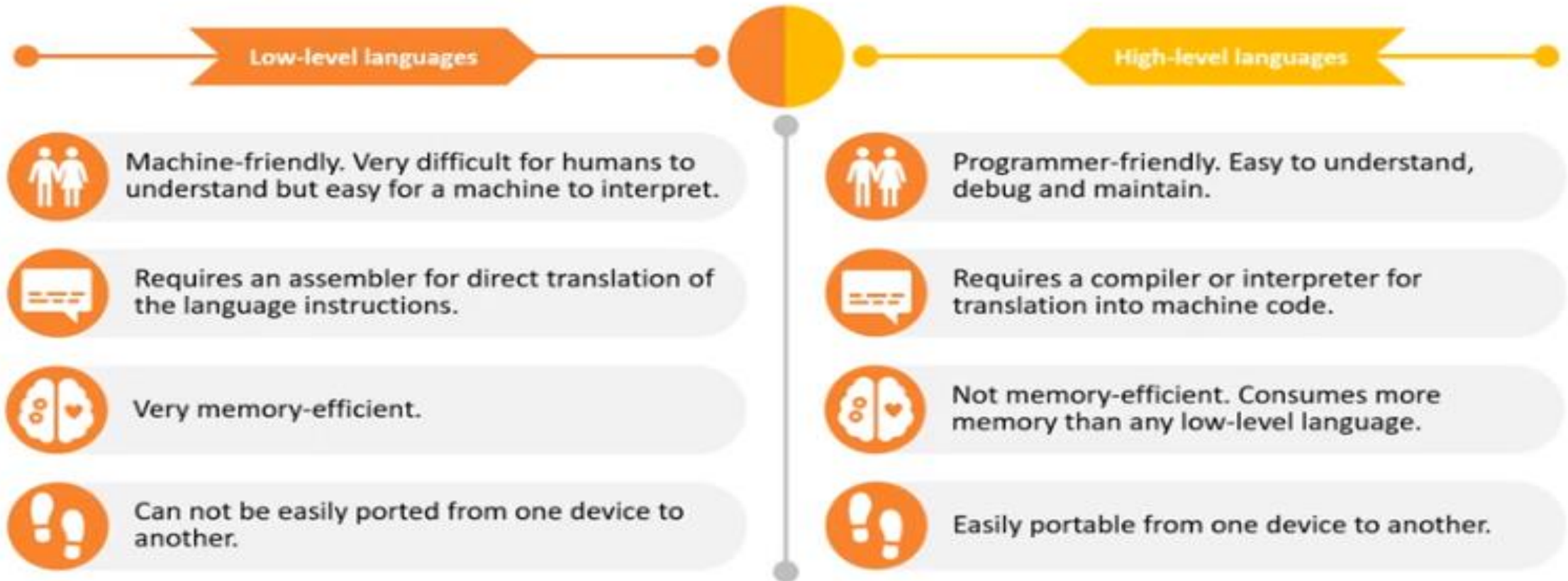
<https://www.youtube.com/watch?v=O8-XzWy2JtA>

The image is a YouTube video player thumbnail. The main title card on the left has an orange background with the text 'CAMBRIDGE IGCSE' in white on a black bar, 'COMPUTER SCIENCE' in white on an orange bar, and '0478 | 0984' in white on a yellow bar. Below this, the text '4.2 HIGH-LEVEL & LOW-LEVEL LANGUAGES' is displayed in large, bold, black letters. On the right side of the thumbnail, there is a preview of a code editor with a dark background and light-colored text. The code appears to be a list of instructions or a program. At the bottom right of the video player, there is a black box with the white text '5:08', indicating the video's duration.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages

Low-level vs high-level languages



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages

Low-level vs high-level languages



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages

Differences between high-level and low-level languages

Language	Advantages	Disadvantages
High-level	independent of the type of computer being used easier to read, write and understand programs quicker to write programs programs are easier and quicker to debug easier to maintain programs in use	programs can be larger programs can take longer to execute programs may not be able to make use of special hardware
Low-level	can make use of special hardware includes special machine-dependent instructions can write code that doesn't take up much space in primary memory can write code that performs a task very quickly	it takes a longer time to write and debug programs programs are more difficult to understand

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages



High-level and low-level languages

Machine code

- Binary representation of instructions in a format that the CPU can decode and execute.

Low-level languages

- Written in assembly language.
- Converted into machine code using a translator called an assembler.
- Used for embedded systems and device drivers where it is necessary to instruct hardware directly.
- One instruction translates into one machine code instruction.
- Code only works with one specific type of processor.
- Programmers work with memory directly.
- Code is harder to write and understand.
- Memory-efficient.
- Code is quick to execute.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.1: High-level languages and low-level languages



High-level and low-level languages

High-level languages

- Written in languages like Python, C++, Java and Visual Basic.
- Converted into machine code using a translator called a compiler or interpreter.
- Makes writing programs easier by utilising commands that are like English.
- One source code instruction translates to many machine code instructions.
- Code will run on different types of processors.
- Programmers can use many different data structures.
- Code is quicker and easier to understand and write.
- Less memory-efficient.
- Code can be slower to execute if it is not optimised.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.2: Assembly languages

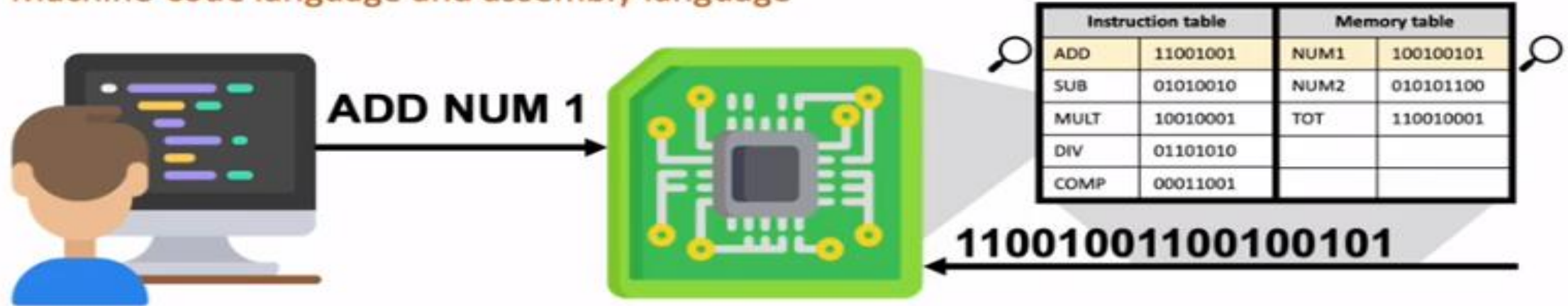
<https://www.youtube.com/watch?v=ZcvcrLyQTVs>



4.2 Types of programming language, translators and integrated development environments (IDEs)

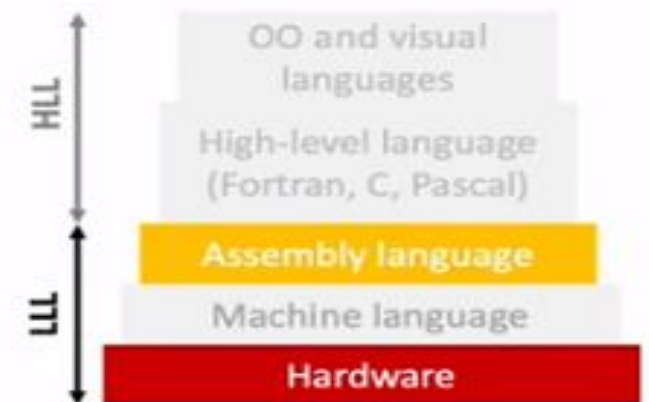
4.2.2: Assembly languages

Machine-code language and assembly language



Assembly language

- Programming using short-code words known as **mnemonics**.
- The **mnemonic** is received by the computer and looked up in tables.
- If a match is found, the word is replaced with a sequence of binary 1s and 0s.
- One-to-one relationship.
- Written in **assembly language** and translated by a specific **assembler**.

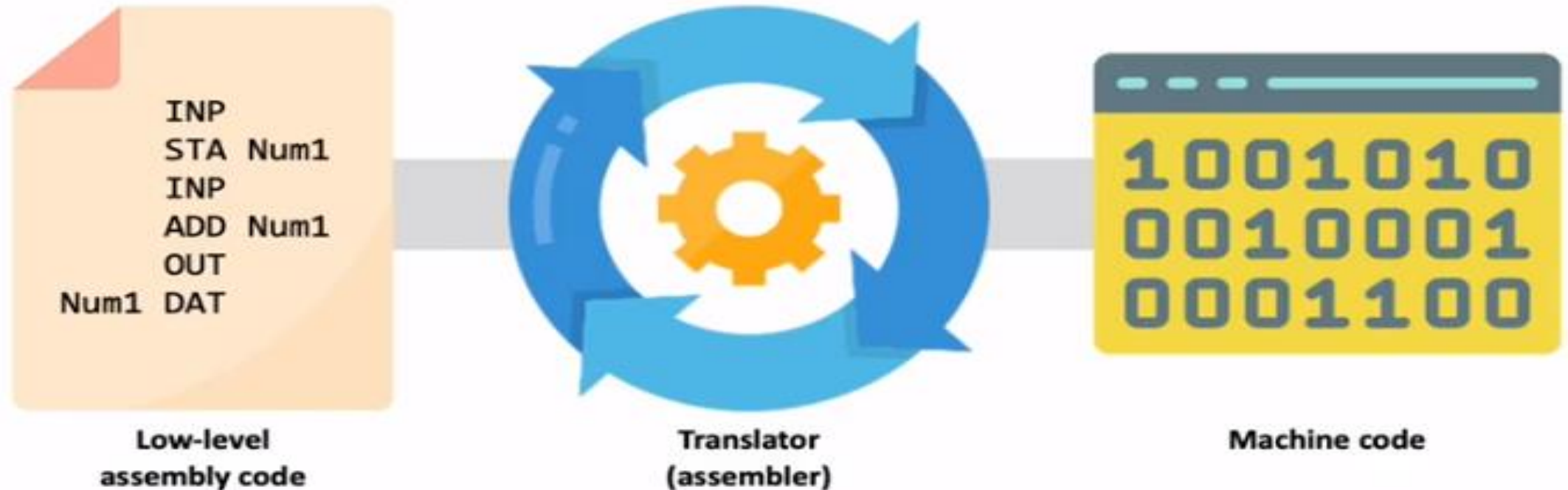


4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.2: Assembly languages

What is assembly language?

Assembly language is directly translated into binary **machine code** by an **assembler**.



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.2: Assembly languages



Assembly language

Machine code

- Binary representation of instructions in a format that the CPU can decode and execute.

Low-level languages

- Written in **assembly language**.
- A form of **low-level language** that uses **mnemonics**.
- **Mnemonics** are abbreviations of operations, typically using short alphabet codes – e.g., LDA, STA, INP
- Translated to an **assembler** into **machine code**.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.3: Compilers and interpreters

https://www.youtube.com/watch?v=_C5AHaS1mOA



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.3: Compilers and interpreters

<https://www.youtube.com/watch?v=c1nisKE8ln4>



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.3: Compilers and interpreters

Translators – Assemblers, compilers and interpreters

	Assembler	Compiler	Interpreter
Description	• Translates assembly language into machine code. • Takes basic commands and operations and converts them into binary code that can be recognised by a specific type of processor. • The translation process is typically a one-to-one process from assembly code to machine code.	• Translates source code from high-level languages into object code and then machine code to be processed by the CPU. • The whole program is translated into machine code before it is run.	• Translates source code from high-level languages into machine code, ready to be processed by the CPU. • The program is translated line by line as it is running.
Advantages	• Programs written in machine language can be replaced with mnemonics, which are easier to remember. • Memory-efficient. • Speed of execution is faster. • Hardware-oriented. • Requires fewer instructions to accomplish the same result.	• No need for translation at run-time. • Speed of execution is faster. • Code is usually optimised. • Original source code is kept secret.	• Easy to write source code, as the program will always run, stopping when it finds a syntax error. • Code does not need to be recompiled when code is changed. • It is easy to try out commands when the program has paused after finding an error – this makes interpreted languages easy for beginner programmers to learn to code.
Disadvantages	• Long programs written in such languages cannot be executed on small computers. • It takes a long time to code or write the program, as it is more complex in nature. • Difficult to remember syntax. • Lack of portability between computers of different makes.	• Source code is easier to write in a high-level language, but the program will not run with syntax errors, which can make it more difficult to write code. • Code needs to be recompiled when it is changed. • Designed for a specific type of processor.	• Translation software is required at run-time. • Speed of execution is slower. • Code is not optimised. • Source code is required.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.3: Compilers and interpreters

Differences between Interpreter and Compiler:

Interpreter translates just one statement of the program at a time into machine code.

An interpreter takes very less time to analyze the source code. However, the overall time to execute the process is much slower.

An interpreter does not generate an intermediary code. Hence, an interpreter is highly efficient in terms of its memory.

Keeps translating the program continuously till the first error is confronted. If any error is spotted, it stops working and hence debugging becomes easy.

Interpreters are used by programming languages like Ruby and Python for example.

Compiler scans the entire program and translates the whole of it into machine code at once.

A compiler takes a lot of time to analyze the source code. However, the overall time taken to execute the process is much faster.

A compiler always generates an intermediary object code. It will need further linking. Hence more memory is needed.

A compiler generates the error message only after it scans the complete program and hence debugging is relatively harder while working with a compiler.

Compilers are used by programming languages like C and C++ for example.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.3: Compilers and interpreters

Compilers and interpreters

Compilers and interpreters both translate **high-level language source code** into **machine code**.



Compiler

- **Translates** the whole code before executing it.
- Produces an executable file.
- Provides error reports for the whole code if errors are detected.
- Mainly used to translate a final program.
- No need for translation at run-time.
- Speed of execution is faster.
- Code is usually optimised.
- Original source code is kept secret.

Interpreter

- **Translates** and executes code line by line.
- Stops executing when an error is found.
- Easy to write source code, as the program will always run, stopping when it finds a syntax error.
- Code does not need to be recompiled when it is changed.
- Mainly used when developing a program.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

https://www.youtube.com/watch?v=kea1M_DUV1o



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

Integrated development environments

IDEs provide the following functions:

- Debugging tools for finding logic errors:
 - Breakpoints – stopping a program at a line of code during execution.
 - Stepping through lines of code one at a time to check which lines are executing.
 - Tracing through a program to output the values of variables.
- Help with preventing and identifying syntax errors:
 - Illustrating keyword syntax and auto-completing command entry.
 - Error highlighting
 - The compiler produces an output of the error message to help identify it.
- Providing a run-time environment:
 - Output window.
 - Simulating different devices the program can run on.
- Usability functions:
 - Navigation, showing/hiding sections of code.
 - Formatting source code.
 - Find and replace.
 - Comment or indent regions.



4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

An integrated development environment (IDE) is a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of at least a source code editor, build automation tools and a debugger. Some IDEs, such as **NetBeans** and **Eclipse**, contain the necessary **compiler, interpreter**.

IDEs usually have these features:

- » code editors
- » a translator
- » a runtime environment with a debugger
- » error diagnostics
- » auto-completion
- » auto-correction
- » an auto-documenter and prettyprinting.

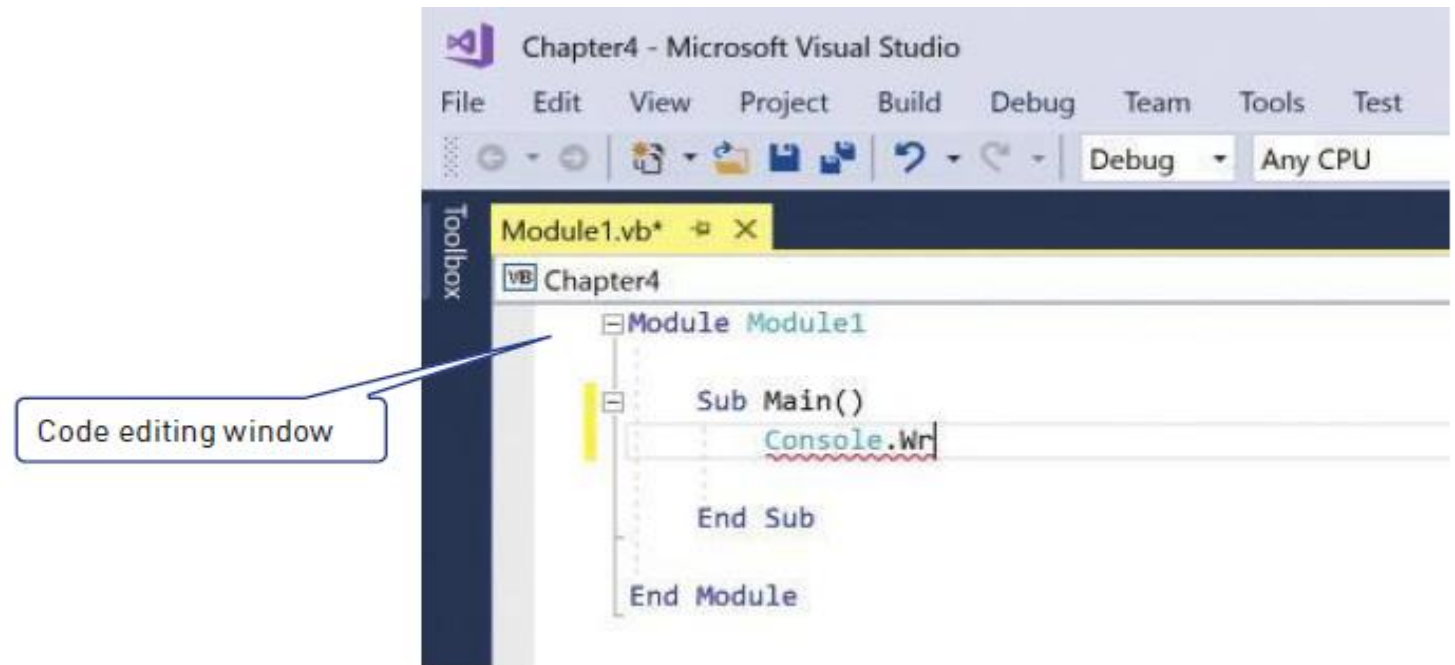
Let's look at each of these features in turn and see how they help the development process.

4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

1 - Code editor:

A code editor allows a program to be written and edited without the need to use a separate text editor. This speeds up the program development process, as editing can be done without changing to a different piece of software each time the program needs correcting or adding to.

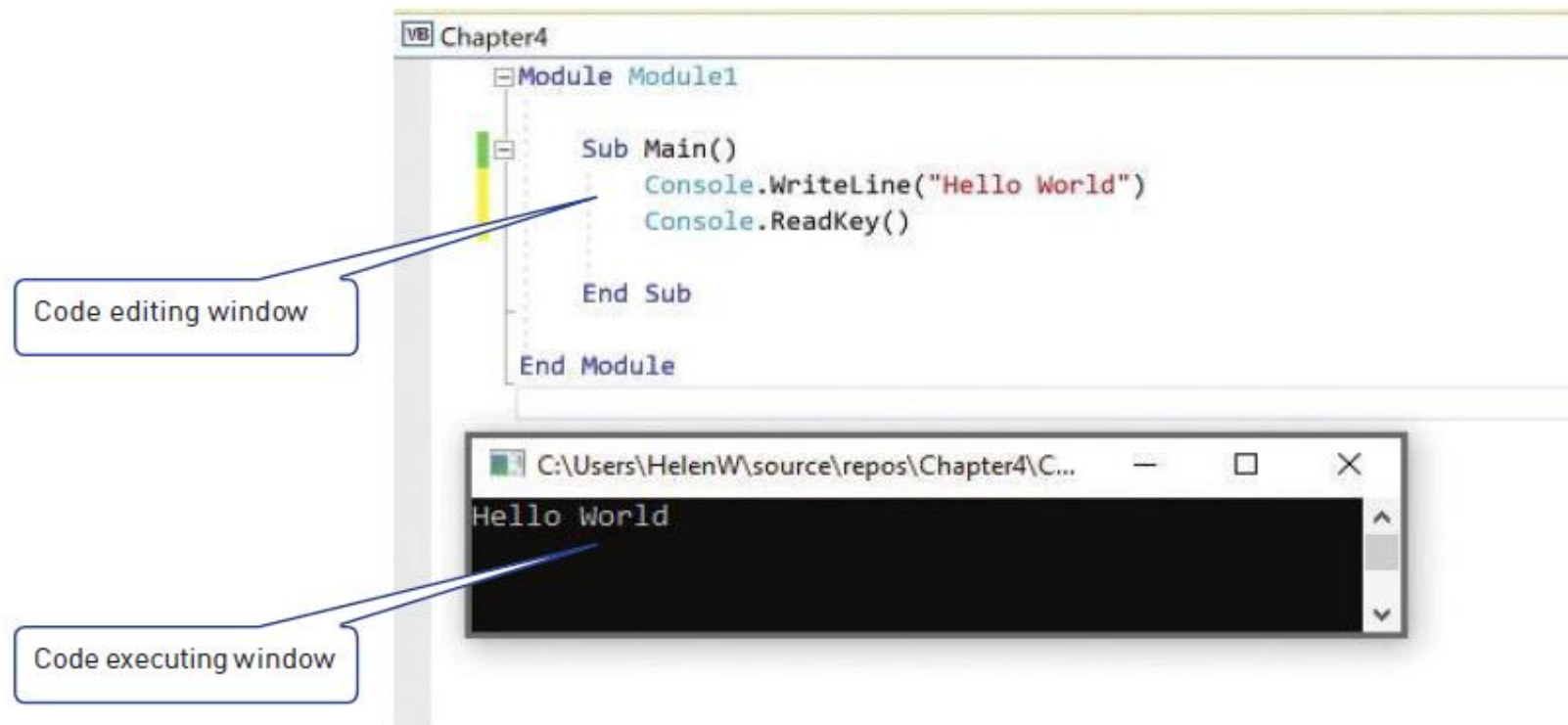


4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

2 - Translator:

Most IDEs usually provide a translator, this can be a compiler and/or an interpreter, to enable the program to be executed. The interpreter is often used for developing the program and the compiler to produce the final version of the program to be used.

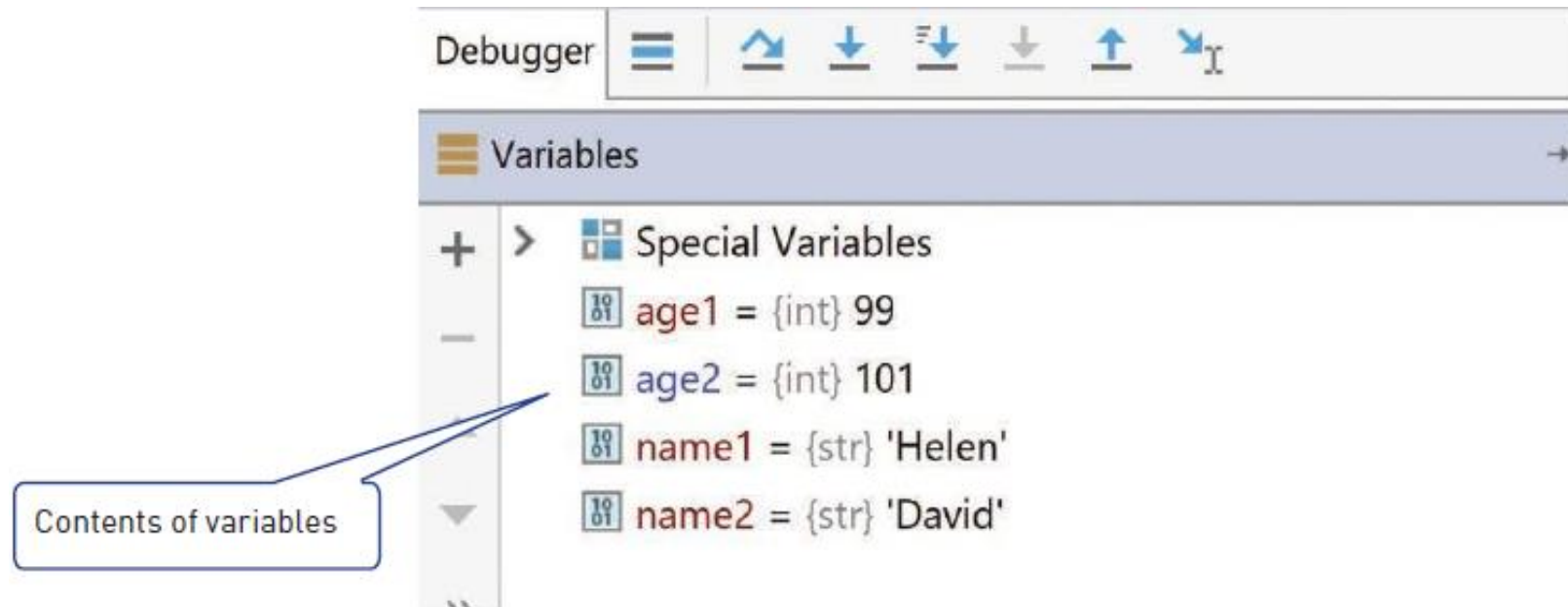


4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

3 - A runtime environment with a debugger:

A debugger is a program that runs the program under development and allows the programmer to step through the program a line at a time (single stepping) or to set a breakpoint to stop the execution of the program at a certain point in the source code. A report window then shows the contents of the variables and expressions evaluated at that point in the program. This allows the programmer to see if there are any logic errors in the program and check that the program works as intended.

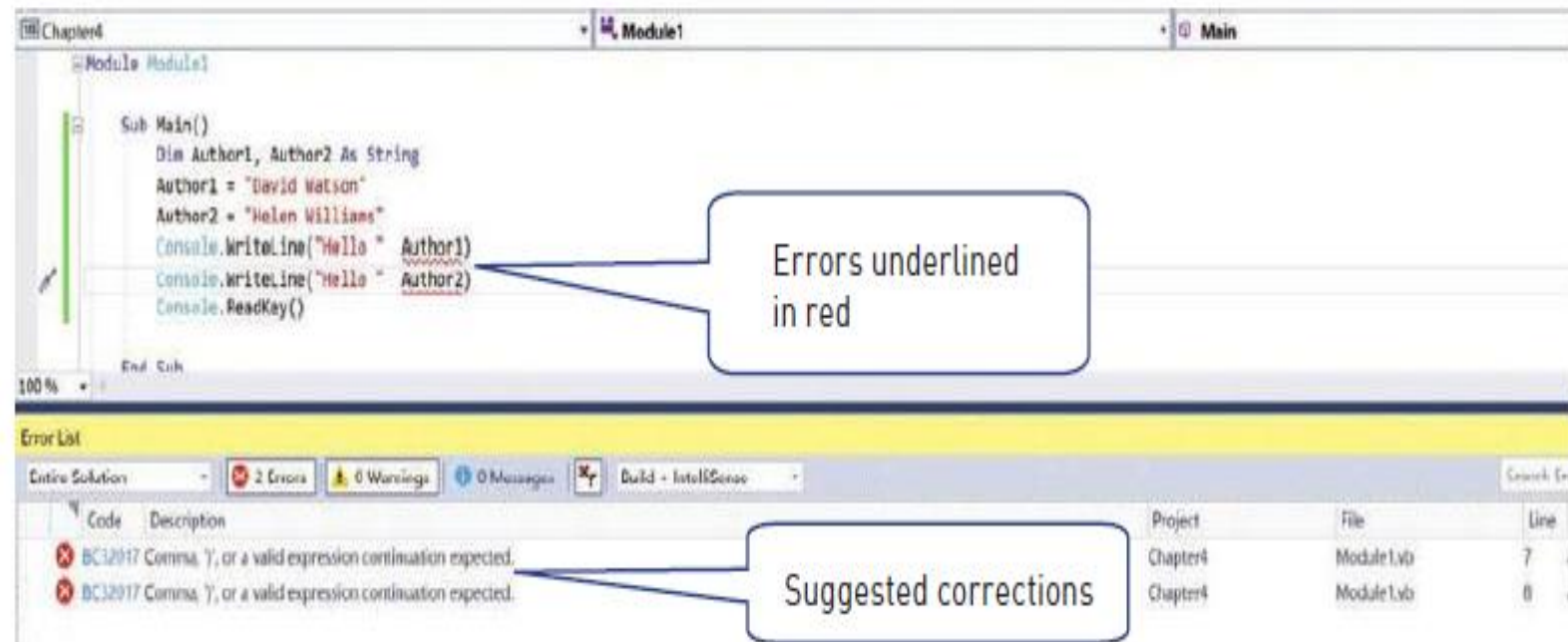


4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

4 - Error diagnostics and auto-correction:

Dynamic error checking finds possible errors as the program code is being typed, alerts the programmer at the time and provides a suggested correction. Many errors can therefore be found and corrected during program writing and editing before the program is run.

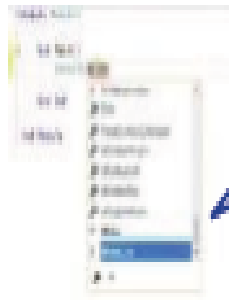


4.2 Types of programming language, translators and integrated development environments (IDEs)

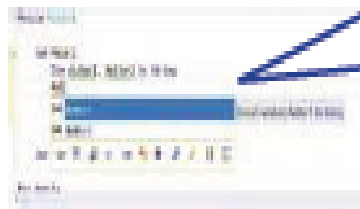
4.2.4: Integrated development environments (IDEs)

5 - Auto-completion:

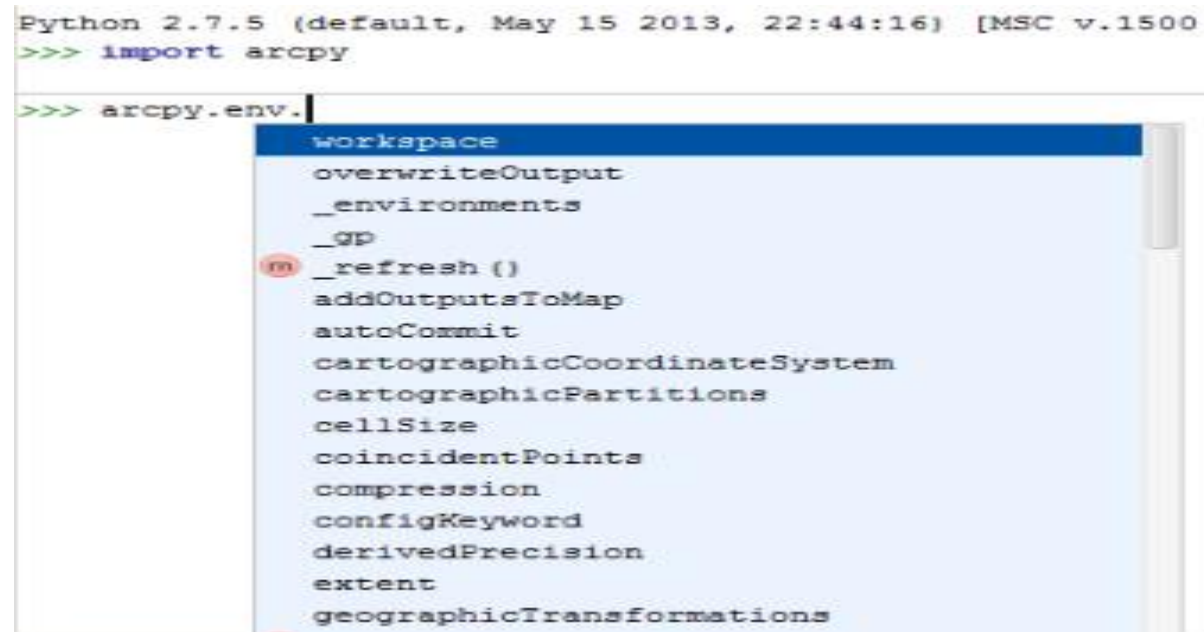
Code editors can offer context-sensitive prompts with text completion for variable names and reserved words.



Auto-completion
Reserved word



Auto-completion
Variable name

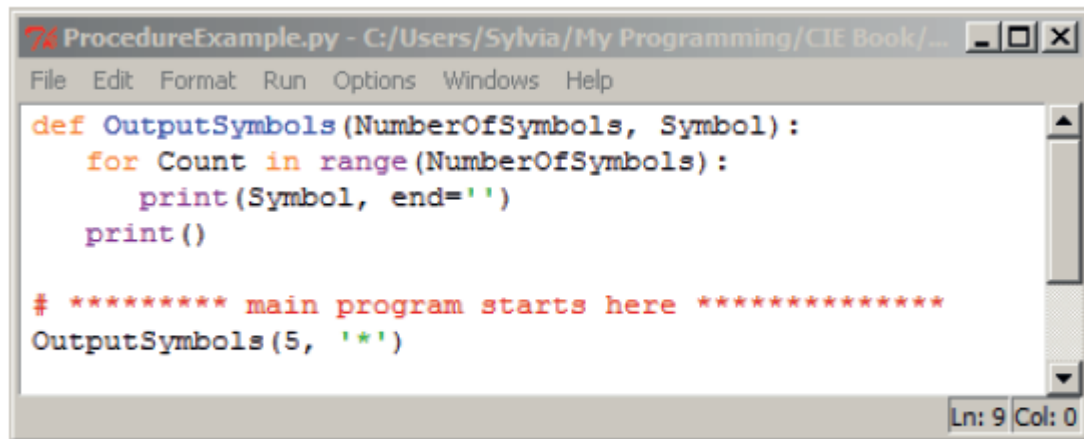


4.2 Types of programming language, translators and integrated development environments (IDEs)

4.2.4: Integrated development environments (IDEs)

6- Prettyprinting:

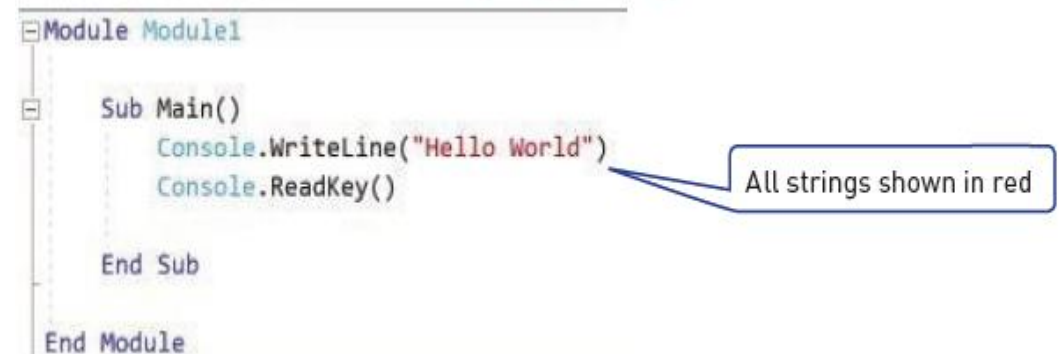
Prettyprint refers to the presentation of the program code typed into an editor. For example, the Python IDLE (see Figure below) automatically colour-codes keywords, built-in function calls, comments, strings and the identifier in a function header. In addition, indentation is automatic.



The screenshot shows the Python IDLE interface with a file named 'ProcedureExample.py'. The code is color-coded: keywords like 'def', 'for', 'in', 'range', 'print', and 'def' are in blue; built-in functions like 'range' and 'print' are in orange; comments are in red; and strings are in green. The code is as follows:

```
def OutputSymbols(NumberOfSymbols, Symbol):  
    for Count in range(NumberOfSymbols):  
        print(Symbol, end='')  
    print()  
  
# ***** main program starts here *****  
OutputSymbols(5, '*')
```

Figure 8.02 Prettyprint in the Python IDLE



The diagram shows a code structure with a callout box pointing to the string 'Hello World' in the 'Console.WriteLine' statement. The callout box contains the text 'All strings shown in red'. The code structure is as follows:

```
Module Module1  
    Sub Main()  
        Console.WriteLine("Hello World")  
        Console.ReadKey()  
    End Sub  
End Module
```